

Neural Networks and Backpropagation

COMP3314
Machine Learning

Dong Xu

Slides adapted from CS231N at Stanford

Neural Networks

Slides adapted from CS231N at Stanford

Neural networks: the original linear classifier

(Before) Linear score function: $f = Wx$

$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$

Neural networks: 2 layers

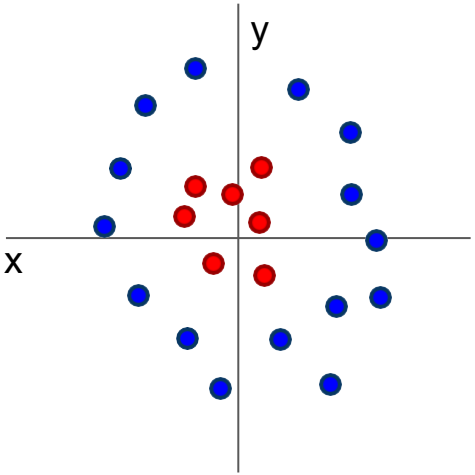
(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$

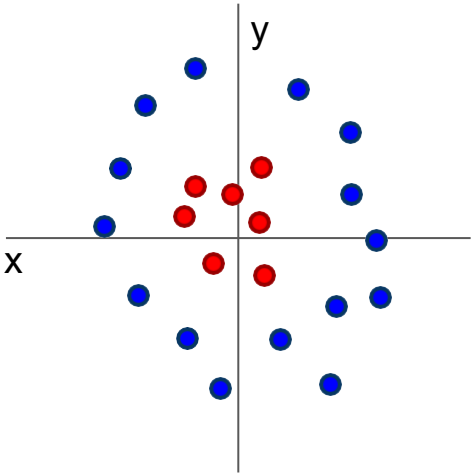
(In practice we will usually add a learnable bias at each layer as well)

Why do we want non-linearity?



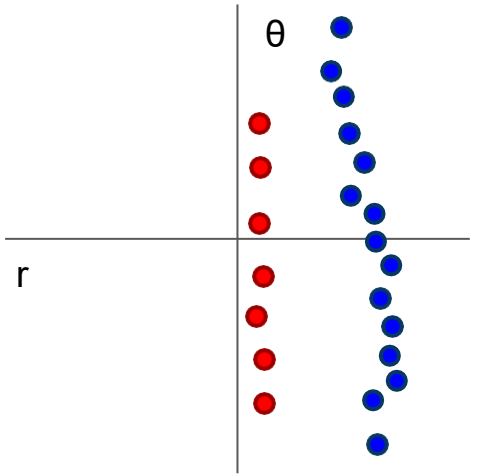
Cannot separate red and blue points with linear classifier

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$



After applying feature transform, points can be separated by linear classifier

Neural networks: also called fully connected network

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

"Neural Network" is a very broad term; these are more accurately called "fully-connected networks" or sometimes "multi-layer perceptrons" (MLP)

(In practice we will usually add a learnable bias at each layer as well)

Neural networks: 3 layers

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network $f = W_3 \max(0, W_2 \max(0, W_1 x))$

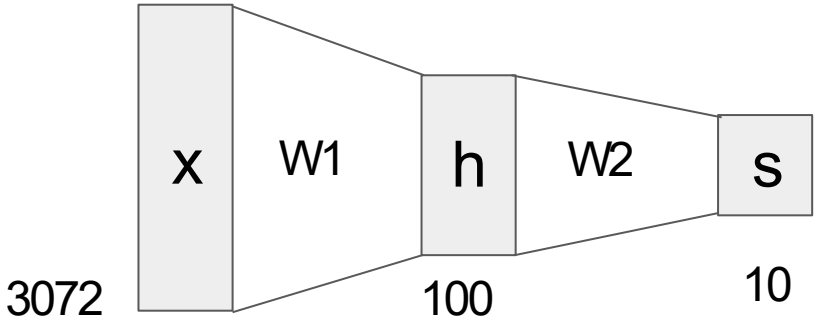
$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H_1 \times D}, W_2 \in \mathbb{R}^{H_2 \times H_1}, W_3 \in \mathbb{R}^{C \times H_2}$$

(In practice we will usually add a learnable bias at each layer as well)

Neural networks: hierarchical computation

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Neural networks: why is max operator important?

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the activation function.

Q: What if we try to build a neural network without one?

$$f = W_2 W_1 x$$

Neural networks: why is max operator important?

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the activation function.

Q: What if we try to build a neural network without one?

$$f = W_2 W_1 x \qquad W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, f = W_3 x$$

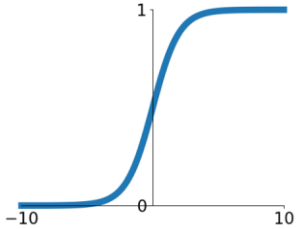
A: We end up with a linear classifier again!

Activation functions

ReLU is a good default choice for most problems

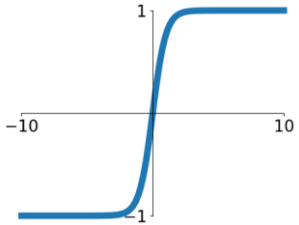
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



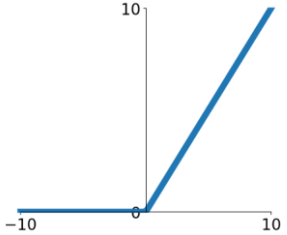
tanh

$$\tanh(x)$$



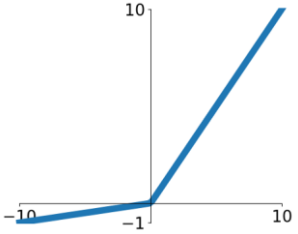
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

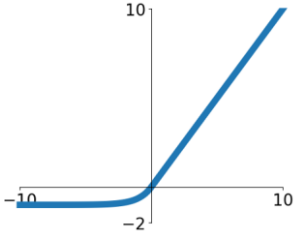


Maxout

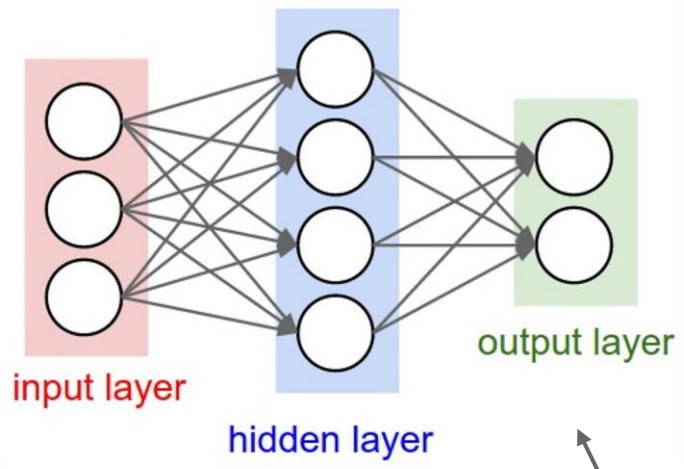
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

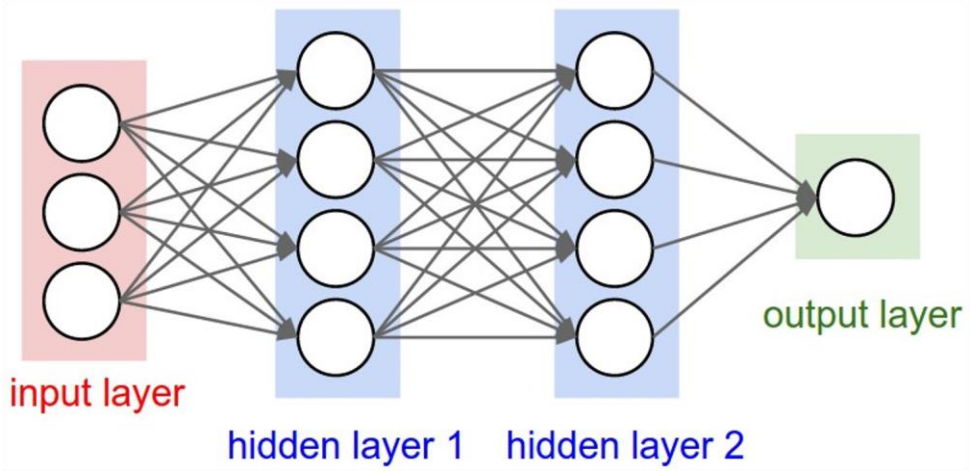
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Neural networks: Architectures





"2-layer Neural Net", or
"1-hidden-layer Neural Net"

"Fully-connected" layers

"3-layer Neural Net", or
"2-hidden-layer Neural Net"

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
import numpy as np
from numpy.random import randn

N, D_in, H, D_out = 64, 1000, 100, 10
x, y = randn(N, D_in), randn(N, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)

for t in range(2000):
  h = 1 / (1 + np.exp(-x.dot(w1)))
  y_pred = h.dot(w2)
  loss = np.square(y_pred - y).sum()
  print(t, loss)

  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h.T.dot(grad_y_pred)
  grad_h = grad_y_pred.dot(w2.T)
  grad_w1 = x.T.dot(grad_h * h * (1 - h))

  w1 -= 1e-4 * grad_w1
  w2 -= 1e-4 * grad_w2
```

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
import numpy as np
from numpy.random import randn

N, D_in, H, D_out = 64, 1000, 100, 10
x, y = randn(N, D_in), randn(N, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)

for t in range(2000):
  h = 1 / (1 + np.exp(-x.dot(w1)))
  y_pred = h.dot(w2)
  loss = np.square(y_pred - y).sum()
  print(t, loss)

  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h.T.dot(grad_y_pred)
  grad_h = grad_y_pred.dot(w2.T)
  grad_w1 = x.T.dot(grad_h * h * (1 - h))

  w1 -= 1e-4 * grad_w1
  w2 -= 1e-4 * grad_w2
```

Define the network

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9    h = 1 / (1 + np.exp(-x.dot(w1)))
10   y_pred = h.dot(w2)
11   loss = np.square(y_pred - y).sum()
12   print(t, loss)
13
14   grad_y_pred = 2.0 * (y_pred - y)
15   grad_w2 = h.T.dot(grad_y_pred)
16   grad_h = grad_y_pred.dot(w2.T)
17   grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19   w1 -= 1e-4 * grad_w1
20   w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9    h = 1 / (1 + np.exp(-x.dot(w1)))
10   y_pred = h.dot(w2)
11   loss = np.square(y_pred - y).sum()
12   print(t, loss)
13
14   grad_y_pred = 2.0 * (y_pred - y)
15   grad_w2 = h.T.dot(grad_y_pred)
16   grad_h = grad_y_pred.dot(w2.T)
17   grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19   w1 -= 1e-4 * grad_w1
20   w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Calculate the analytical gradients

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```
1  import numpy as np
2  from numpy.random import randn
3
4  N, D_in, H, D_out = 64, 1000, 100, 10
5  x, y = randn(N, D_in), randn(N, D_out)
6  w1, w2 = randn(D_in, H), randn(H, D_out)
7
8  for t in range(2000):
9    h = 1 / (1 + np.exp(-x.dot(w1)))
10   y_pred = h.dot(w2)
11   loss = np.square(y_pred - y).sum()
12   print(t, loss)
13
14   grad_y_pred = 2.0 * (y_pred - y)
15   grad_w2 = h.T.dot(grad_y_pred)
16   grad_h = grad_y_pred.dot(w2.T)
17   grad_w1 = x.T.dot(grad_h * h * (1 - h))
18
19   w1 -= 1e-4 * grad_w1
20   w2 -= 1e-4 * grad_w2
```

Define the network

Forward pass

Calculate the analytical gradients

Gradient descent

Plugging in neural networks with loss functions

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$ Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ SVM Loss on predictions

$$R(W) = \sum_k W_k^2$$ Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$ Total loss: data loss + regularization

Problem: How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$ Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ SVM Loss on predictions

$$R(W) = \sum_k W_k^2$$ Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$ Total loss: data loss + regularization

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

(Bad) Idea: Derive $\nabla_W L$ on paper

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

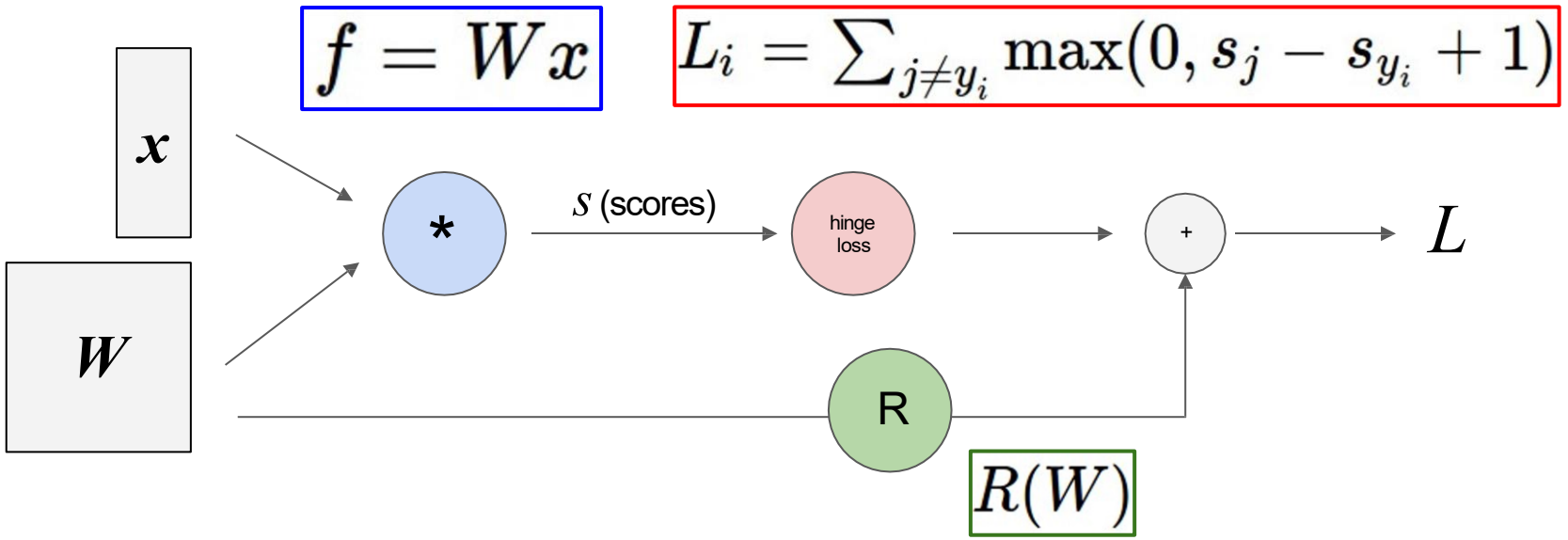
$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

Problem: Very tedious: Lots of matrix calculus, need lots of paper

Problem: What if we want to change loss? E.g. use softmax instead of SVM? Need to re-derive from scratch =(

Problem: Not feasible for very complex models!

Better Idea: Computational graphs + Backpropagation



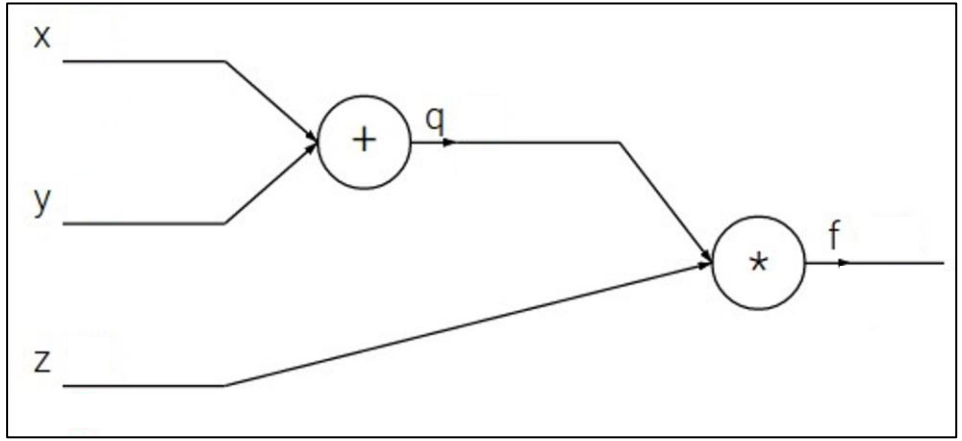
Solution: Backpropagation

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

Backpropagation: a simple example

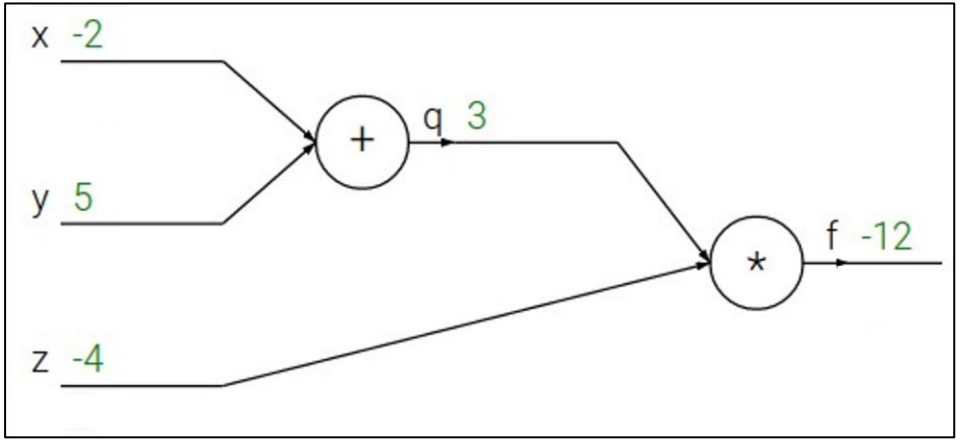
$$f(x, y, z) = (x + y)z$$



Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

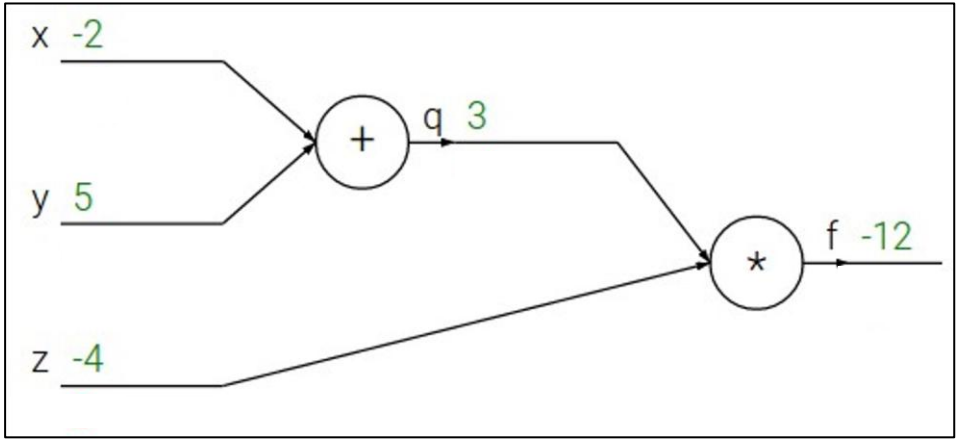


Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



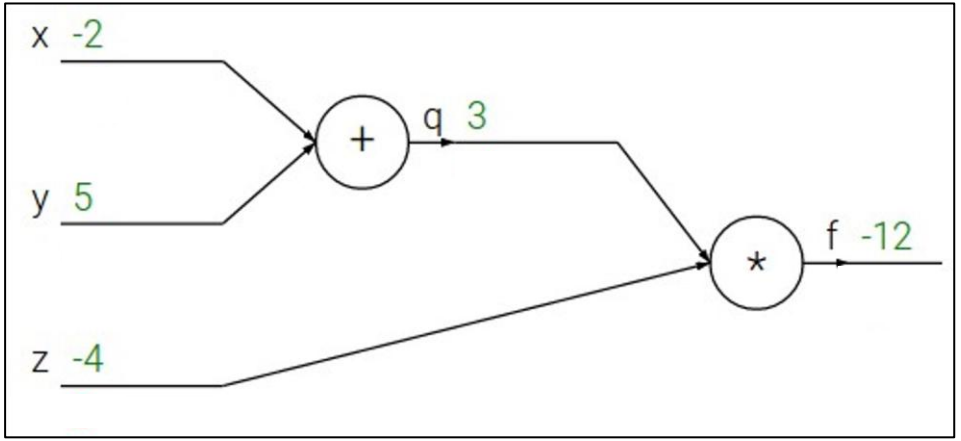
Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



Backpropagation: a simple example

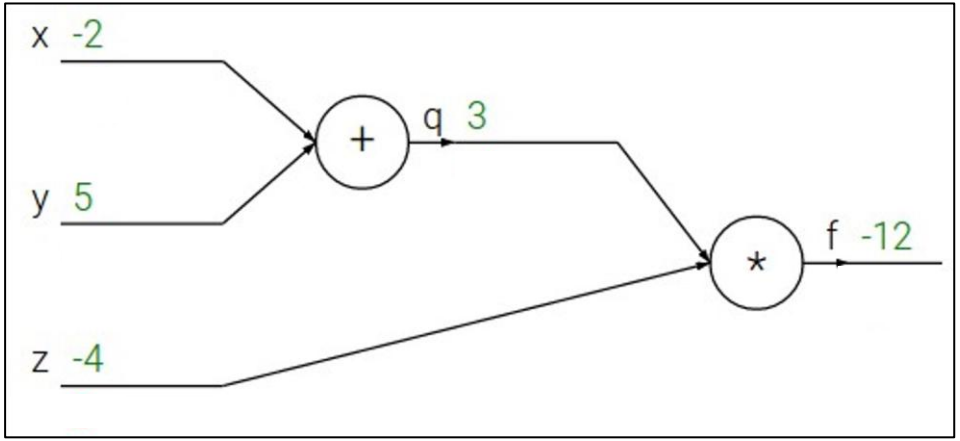
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

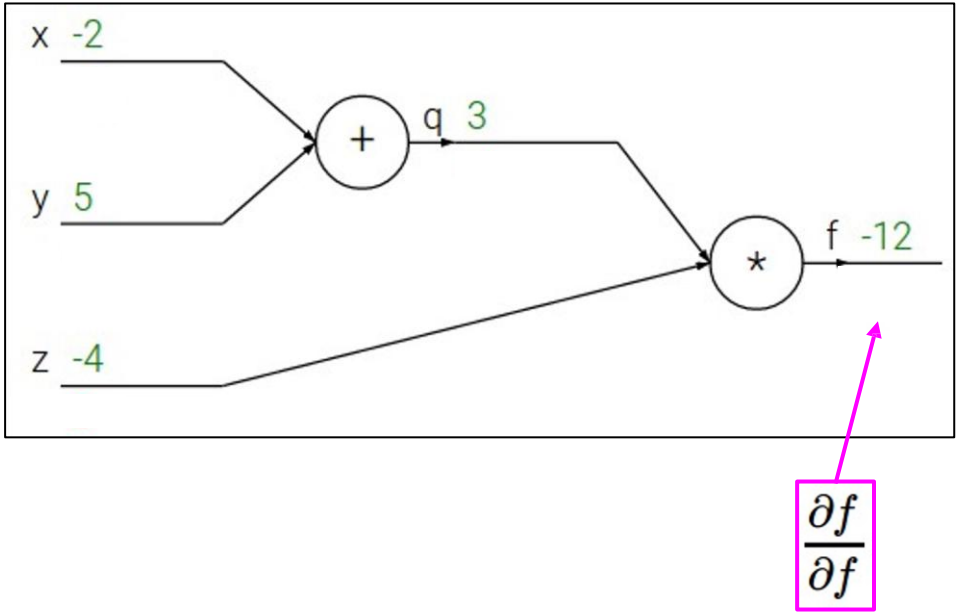
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

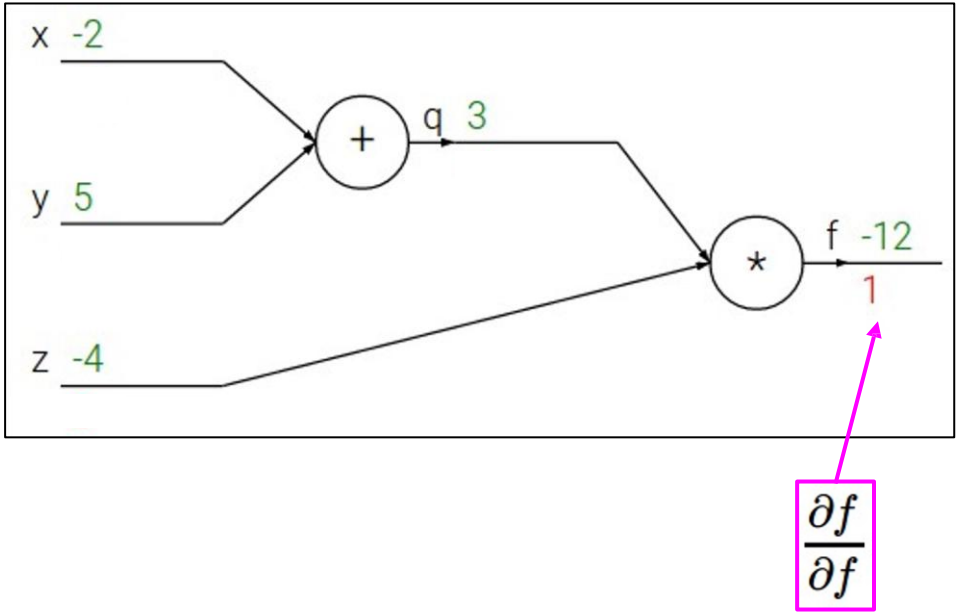
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

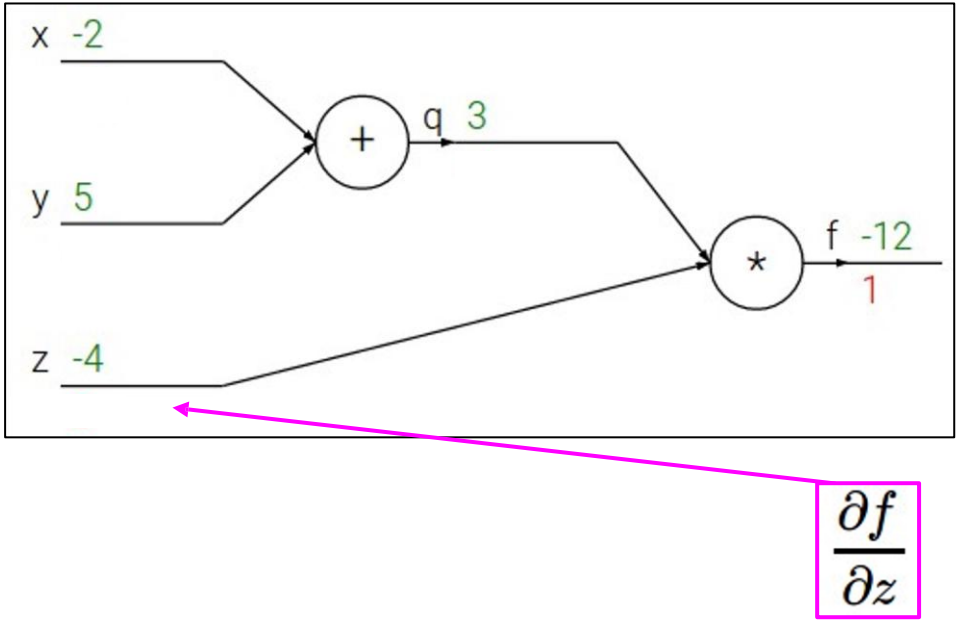
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

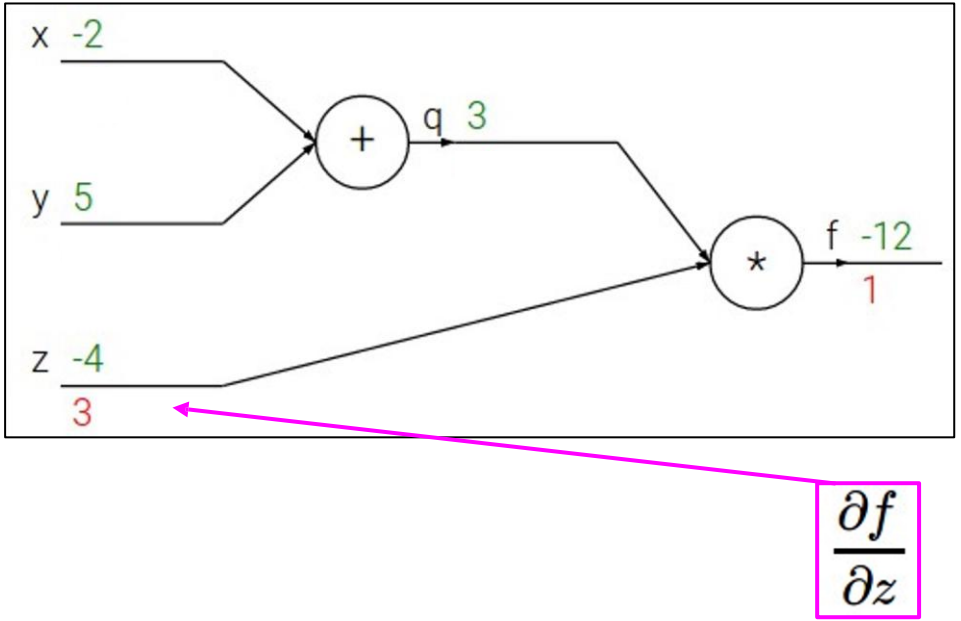
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

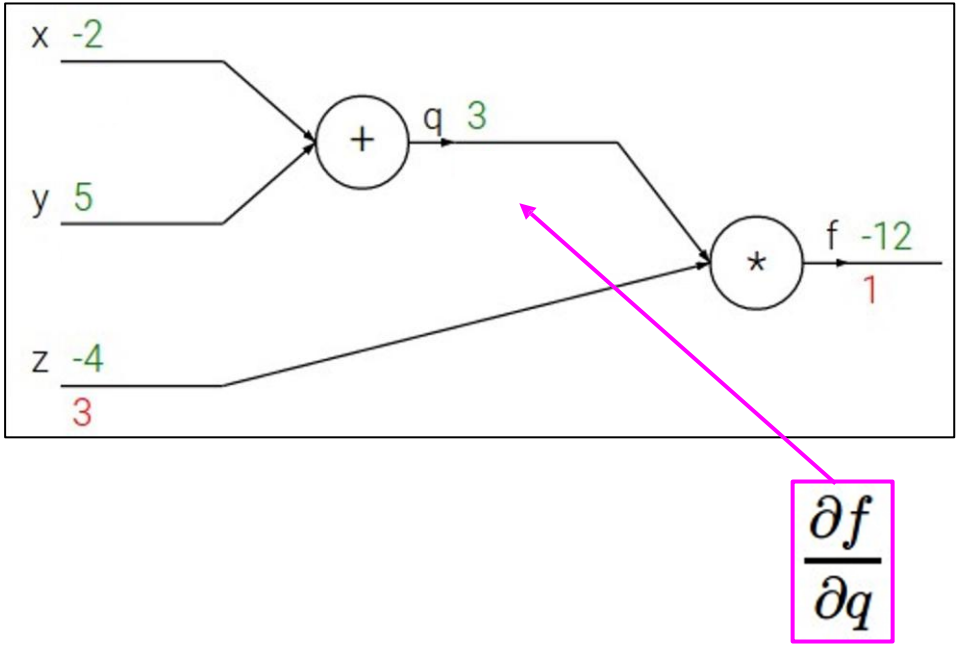
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

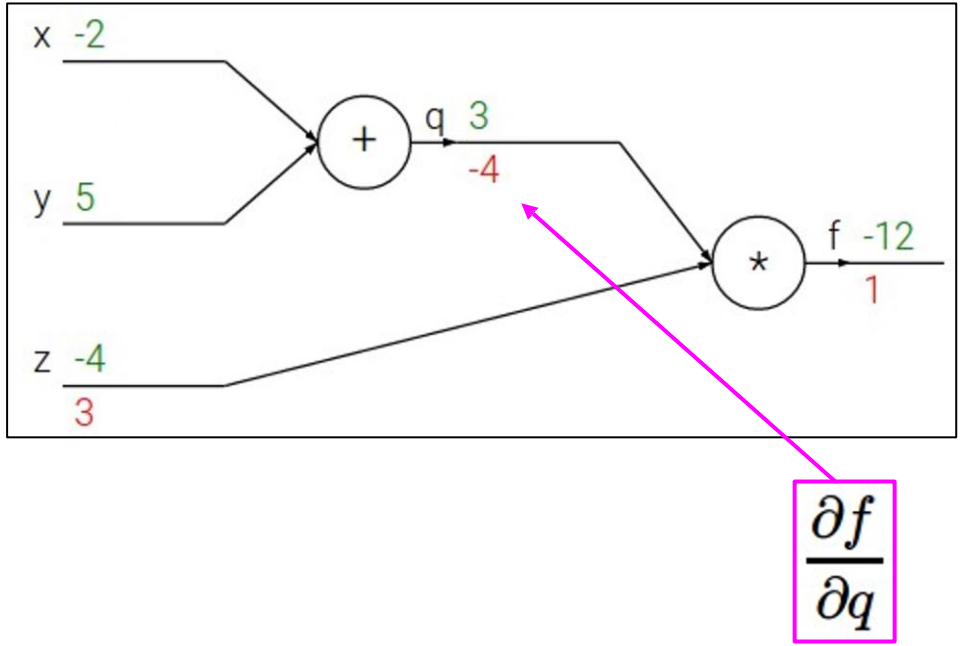
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

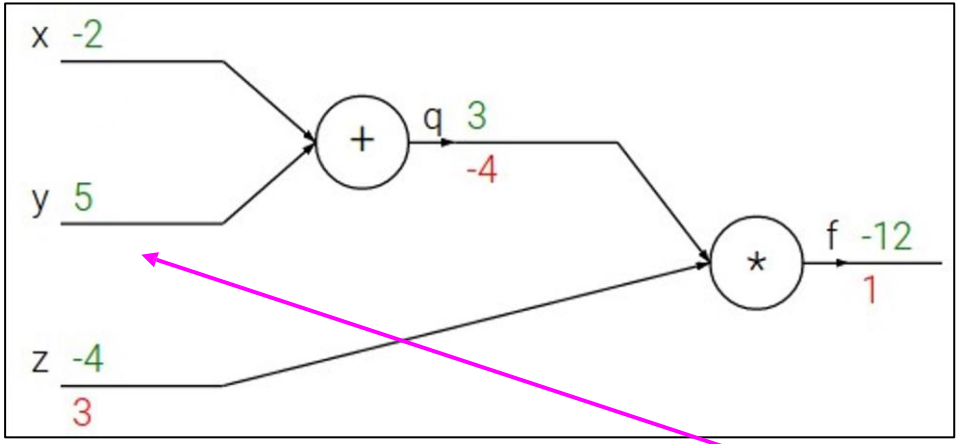
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

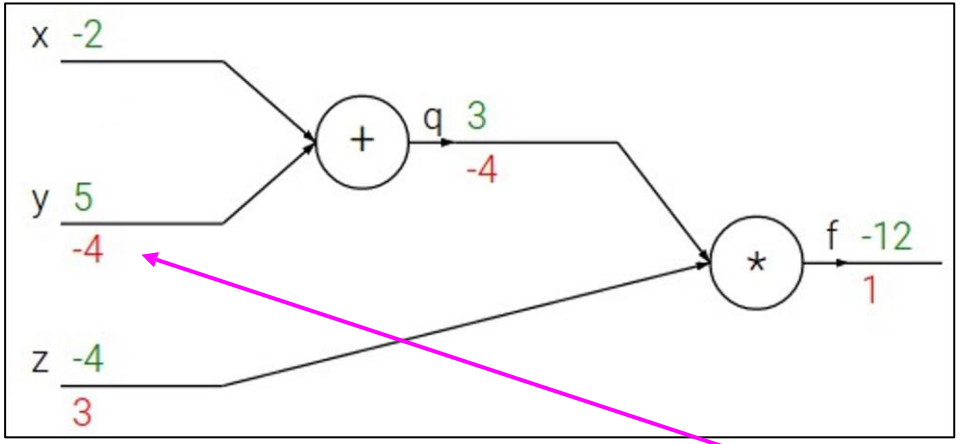
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$\frac{\partial f}{\partial y}$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

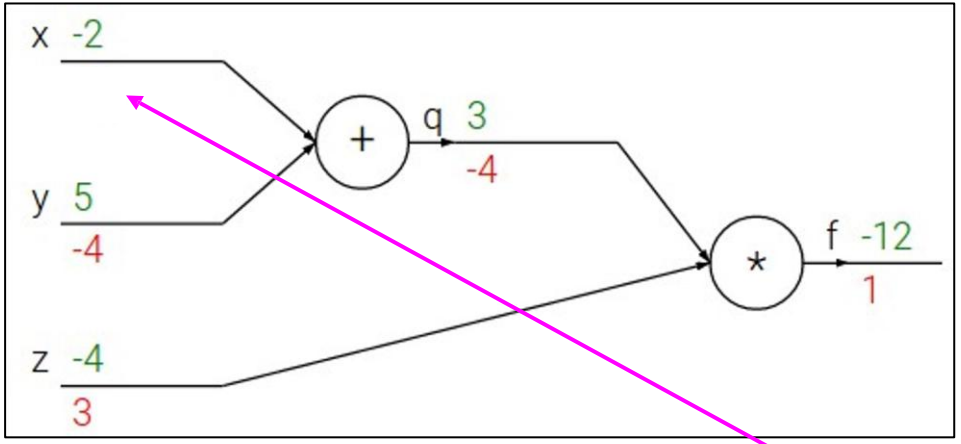
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

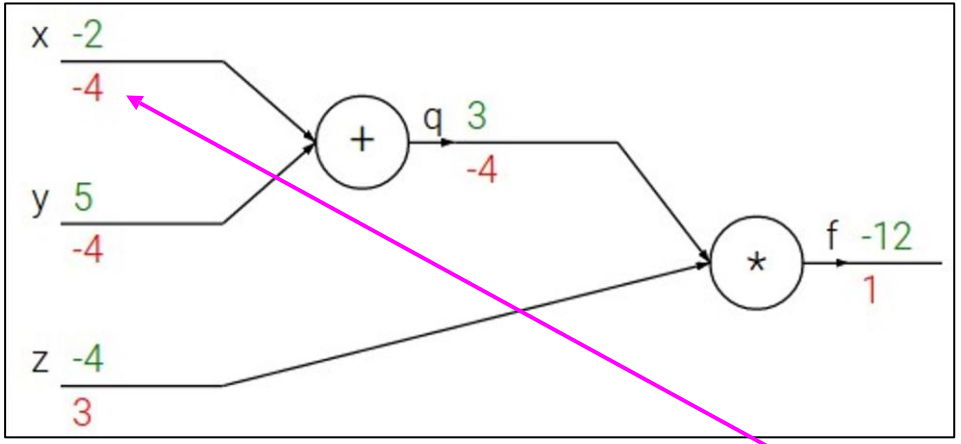
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



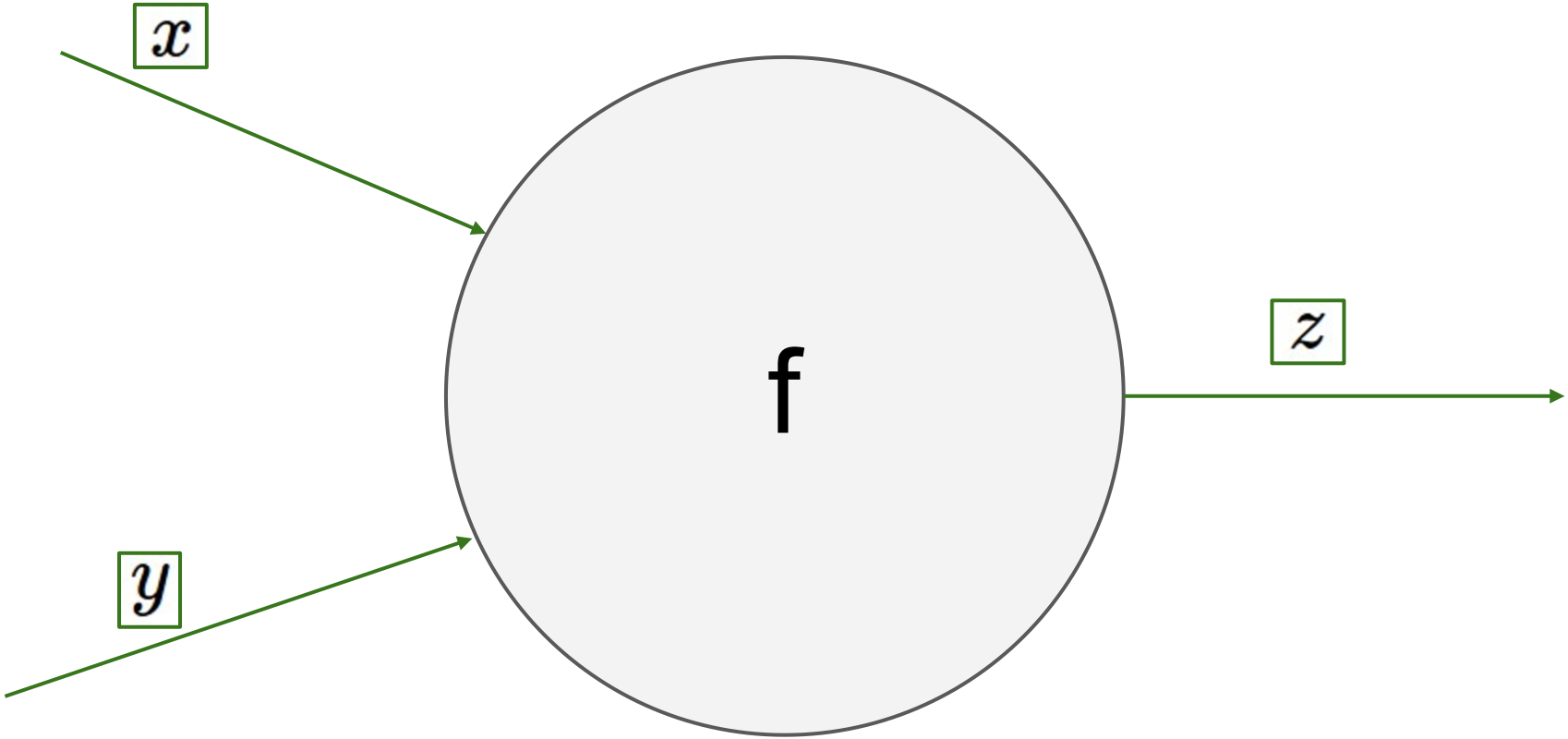
$\frac{\partial f}{\partial x}$

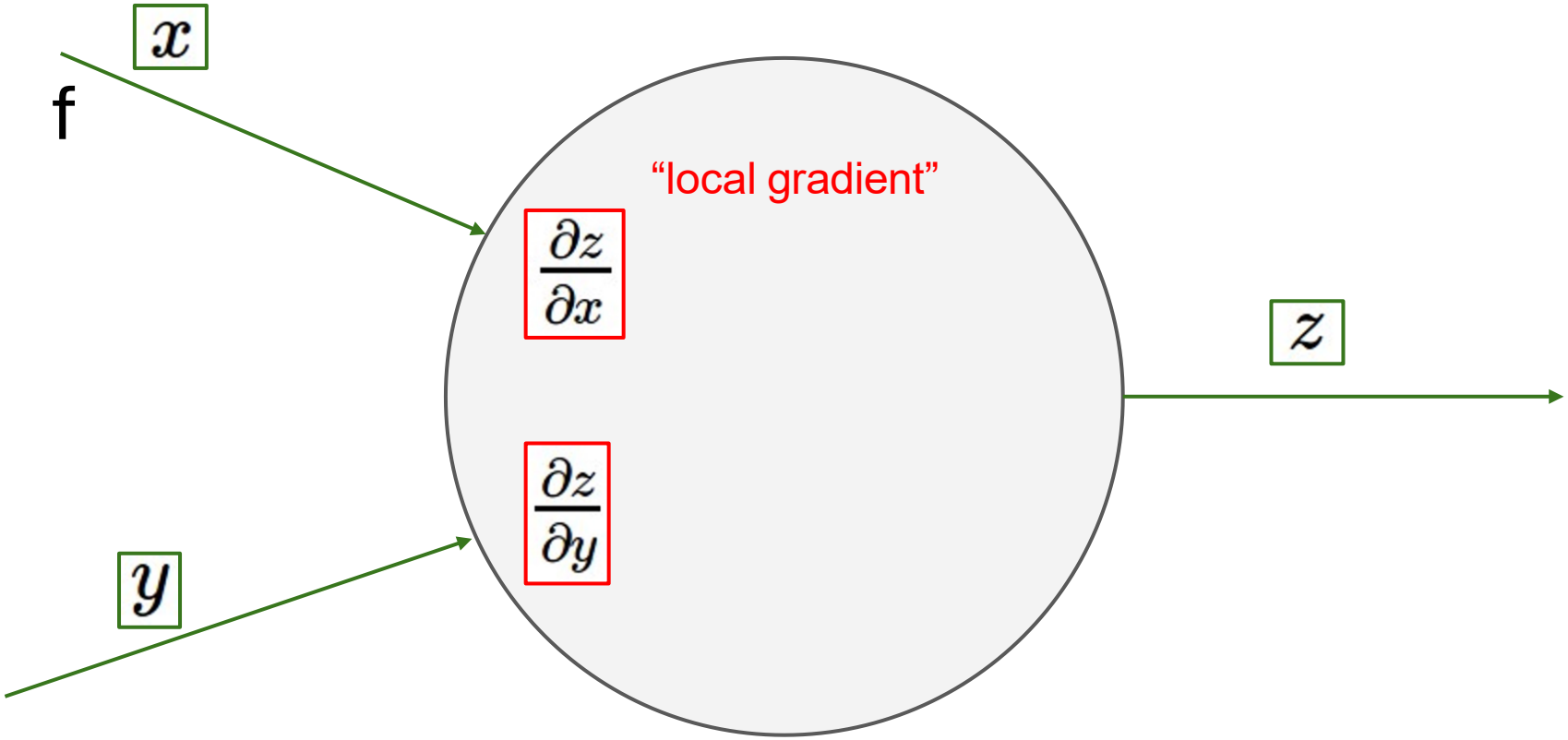
Chain rule:

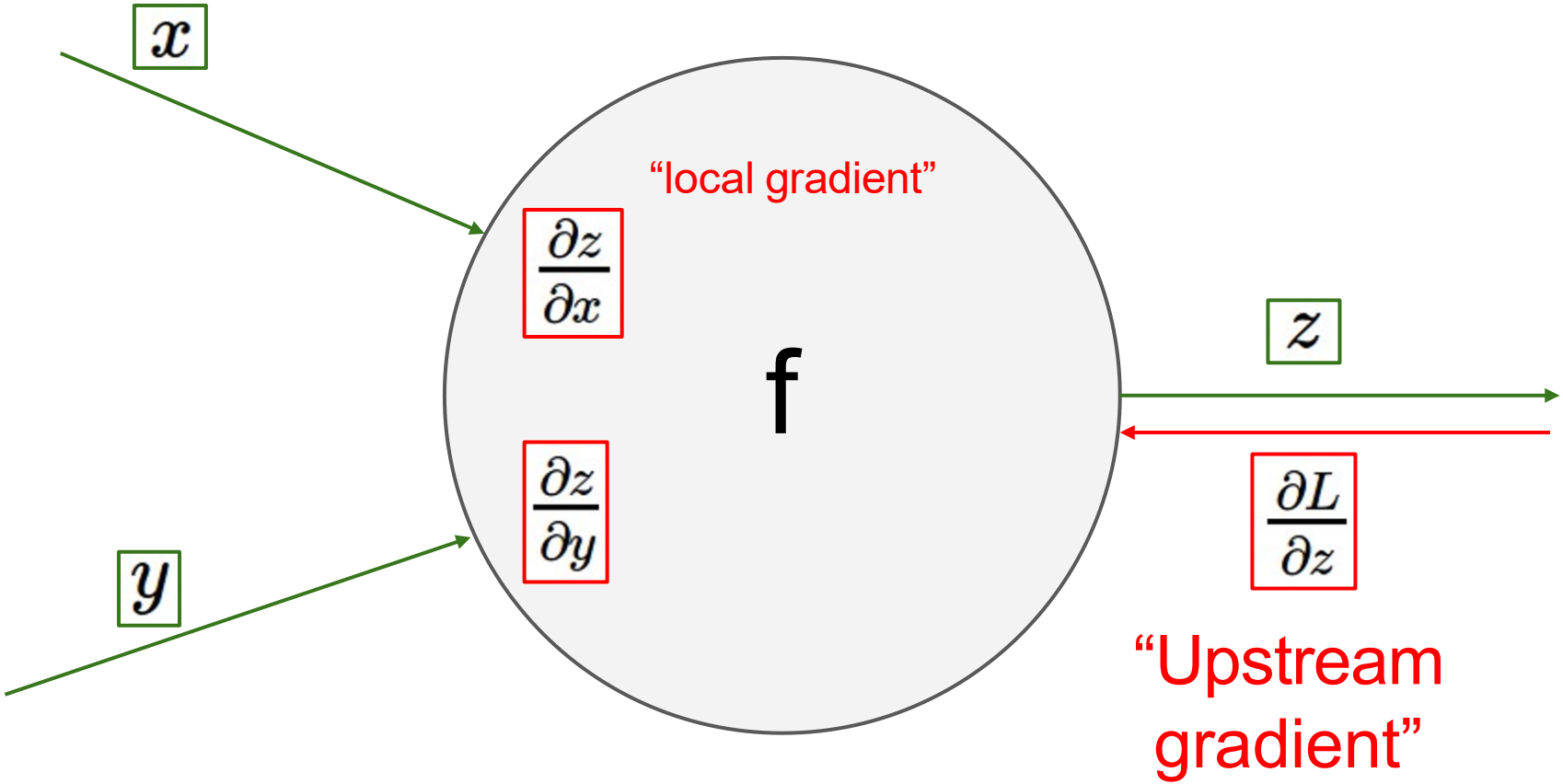
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

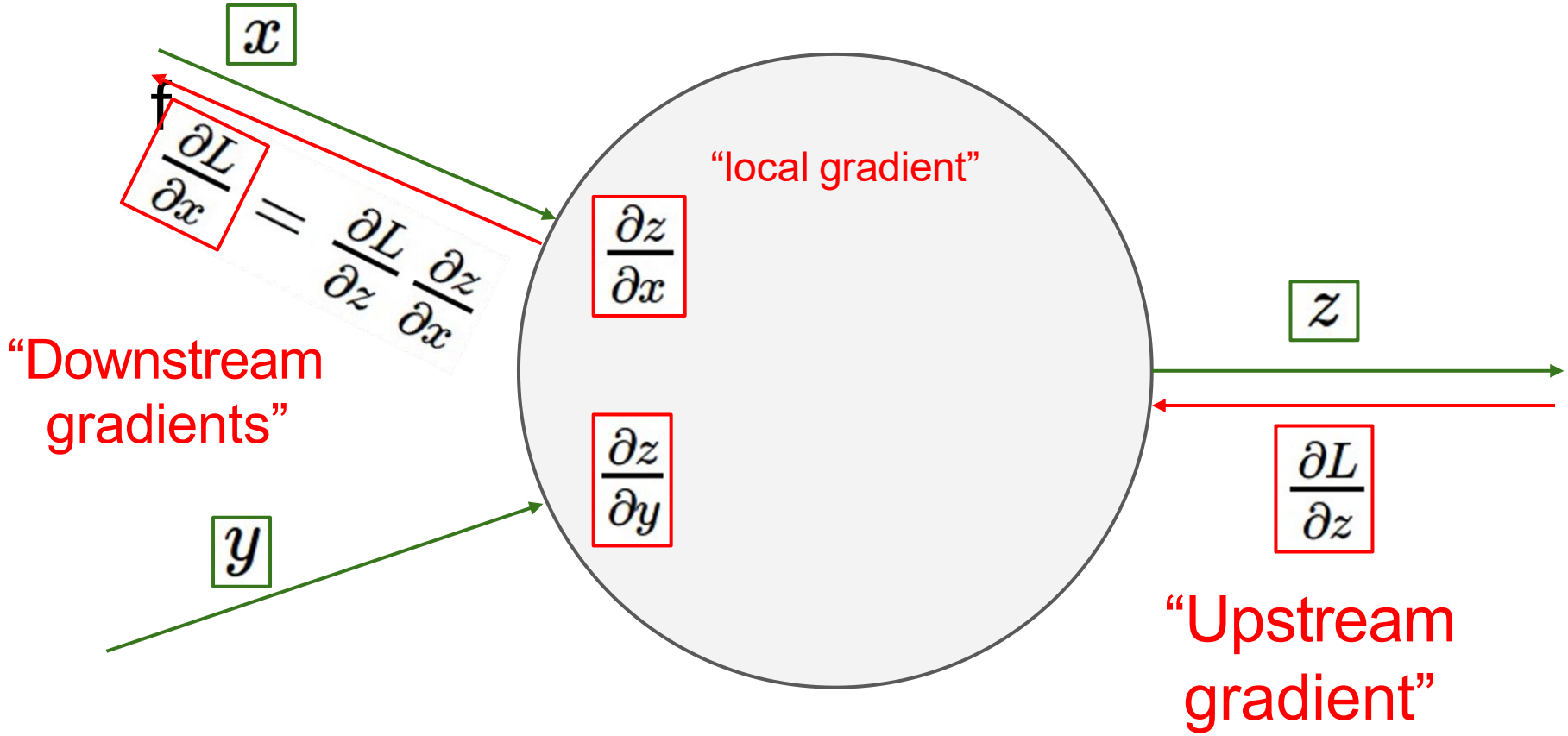
Upstream gradient

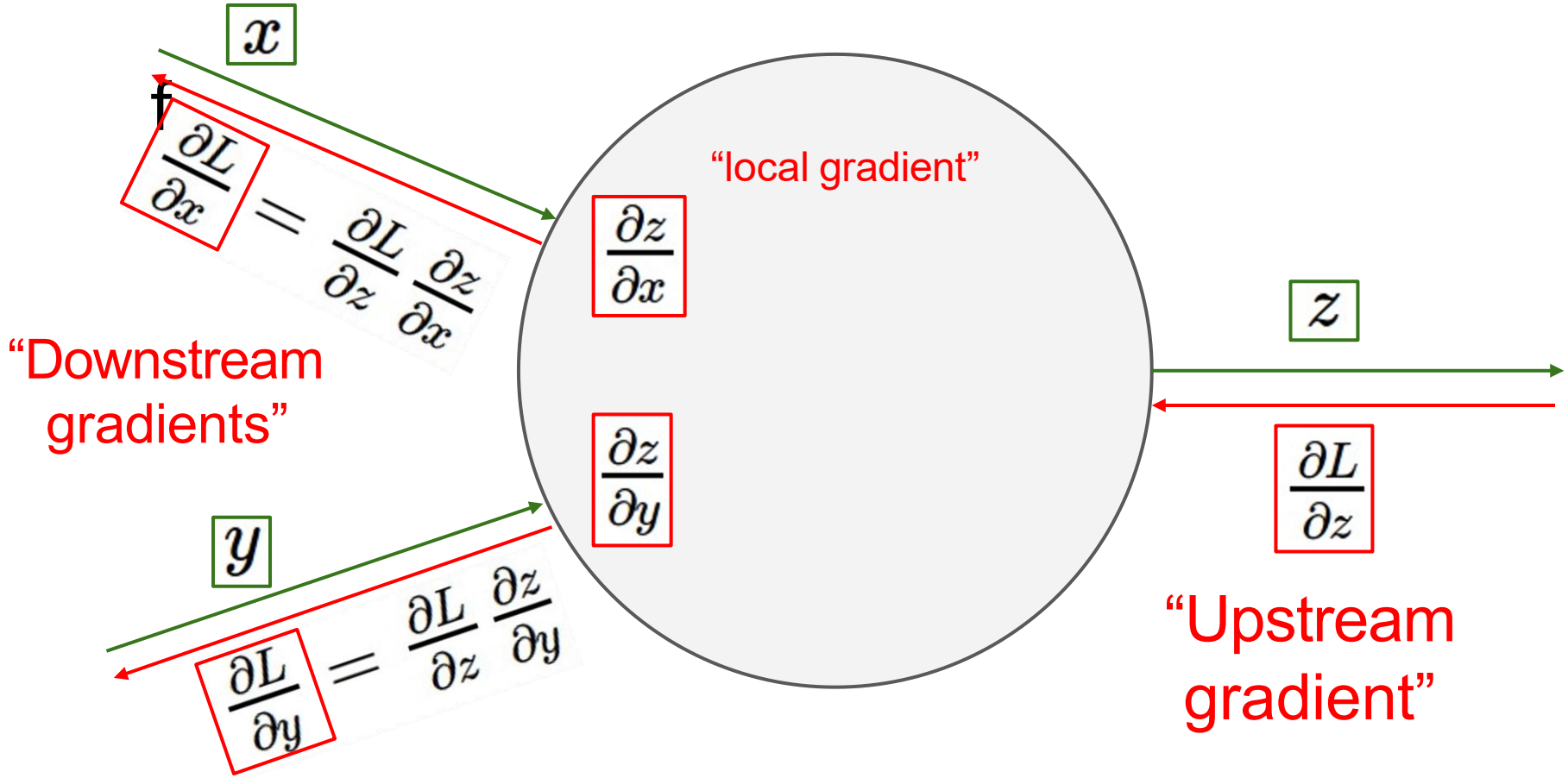
Local gradient

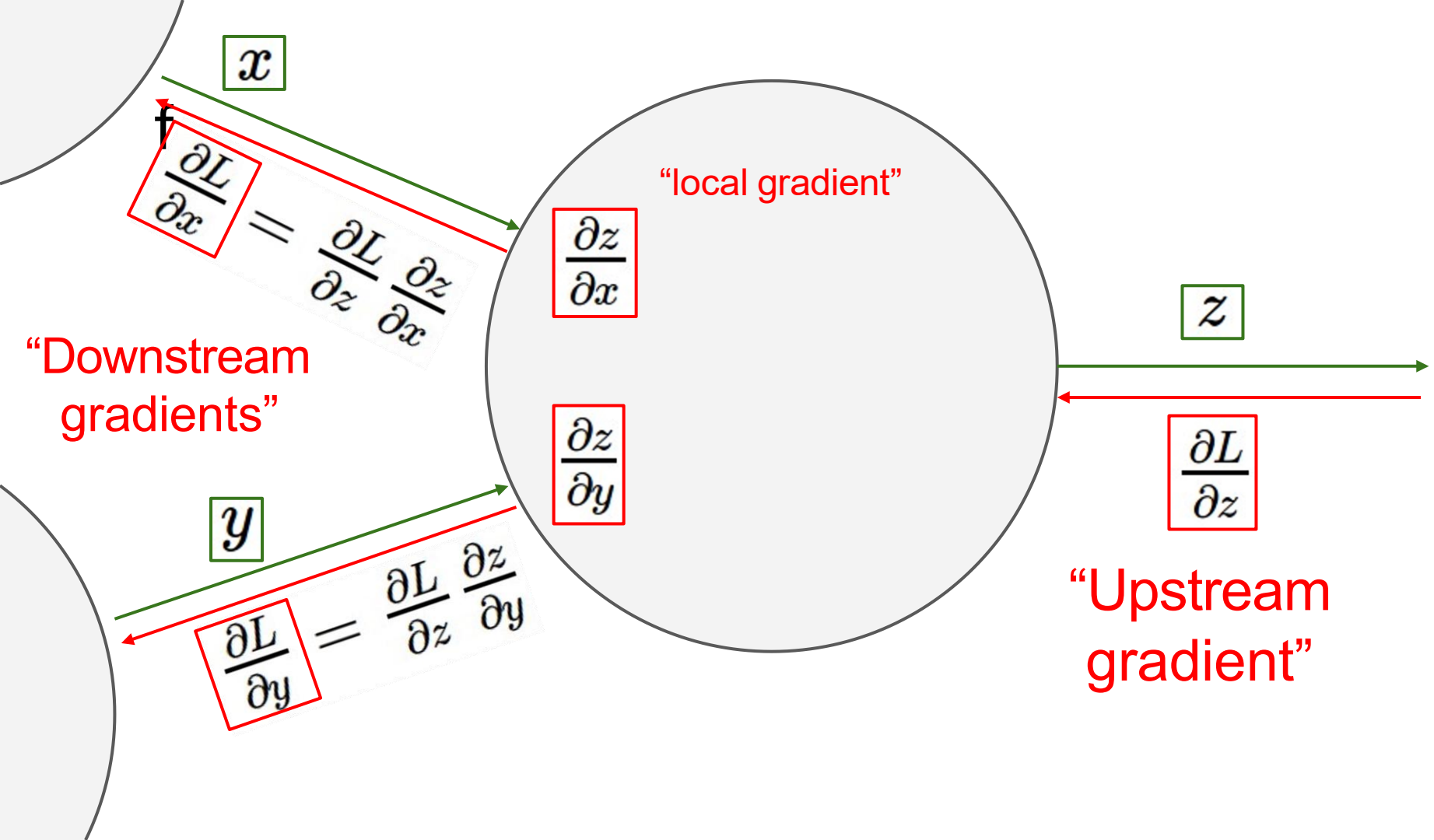

x
y
f
z


x
f
"local gradient"
$\frac{\partial z}{\partial x}$
z
$\frac{\partial z}{\partial y}$
y


x
y
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
"local gradient"
f
z
$\frac{\partial L}{\partial z}$
"Upstream gradient"

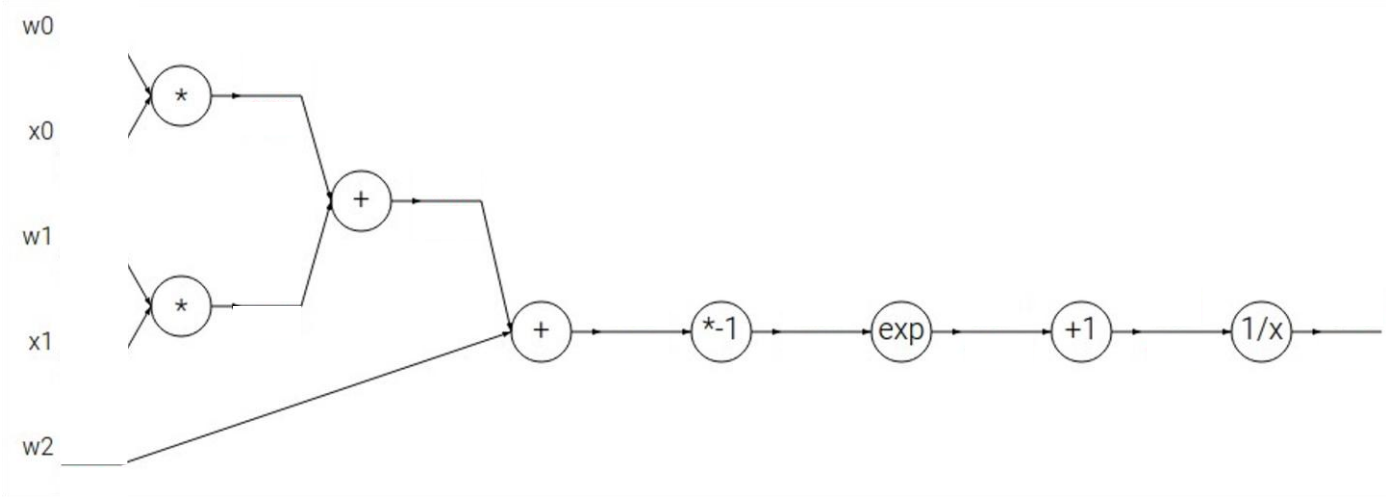

x
y
z
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
$\frac{\partial L}{\partial z}$
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"local gradient"
"Downstream gradients"
"Upstream gradient"


x
y
z
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
"local gradient"
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
$\frac{\partial L}{\partial z}$
"Downstream gradients"
"Upstream gradient"


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"Downstream gradients"
"local gradient"
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
z
$\frac{\partial L}{\partial z}$
"Upstream gradient"
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$

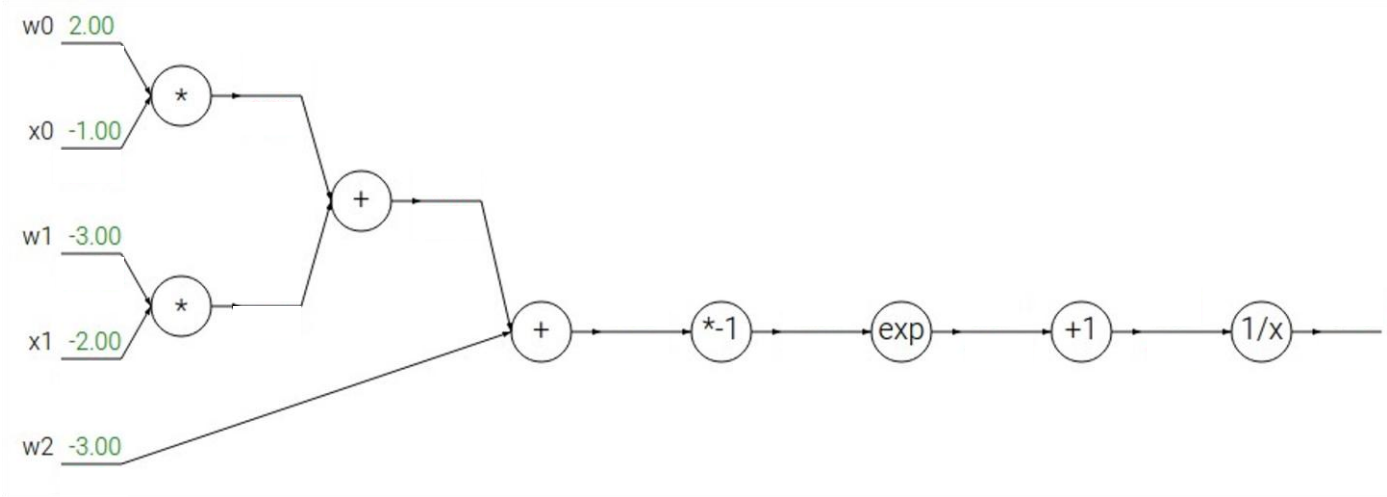
Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



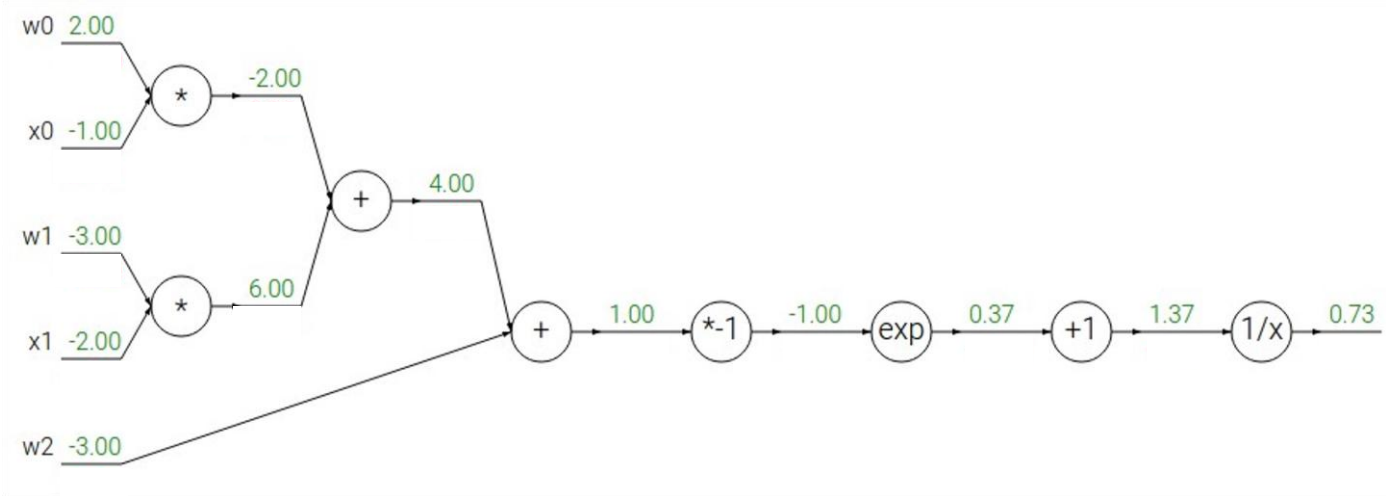
Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

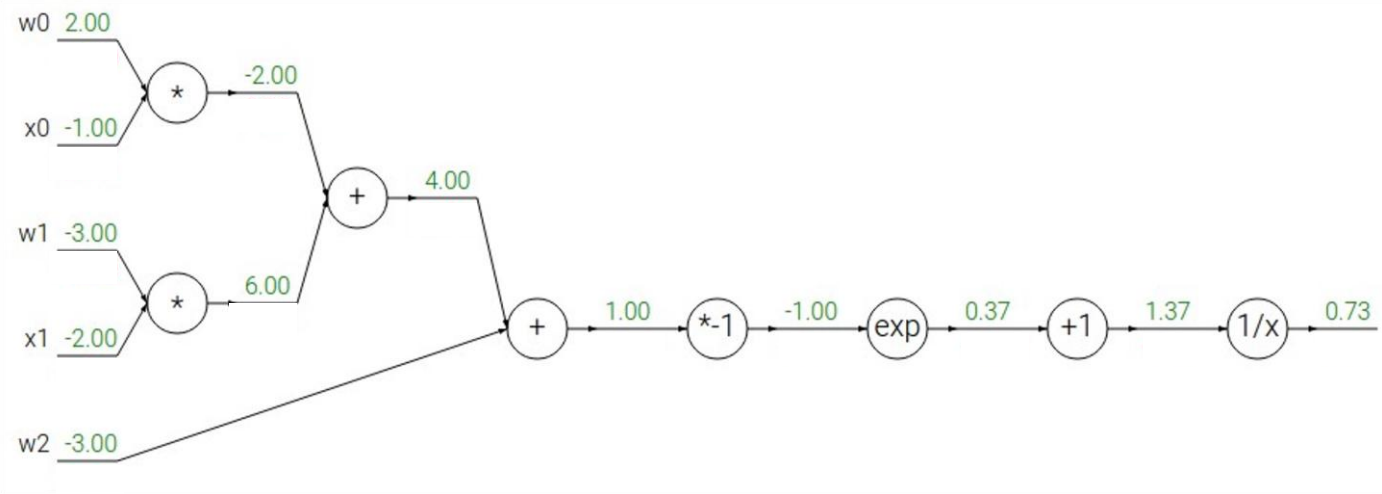


Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



Another example: $$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

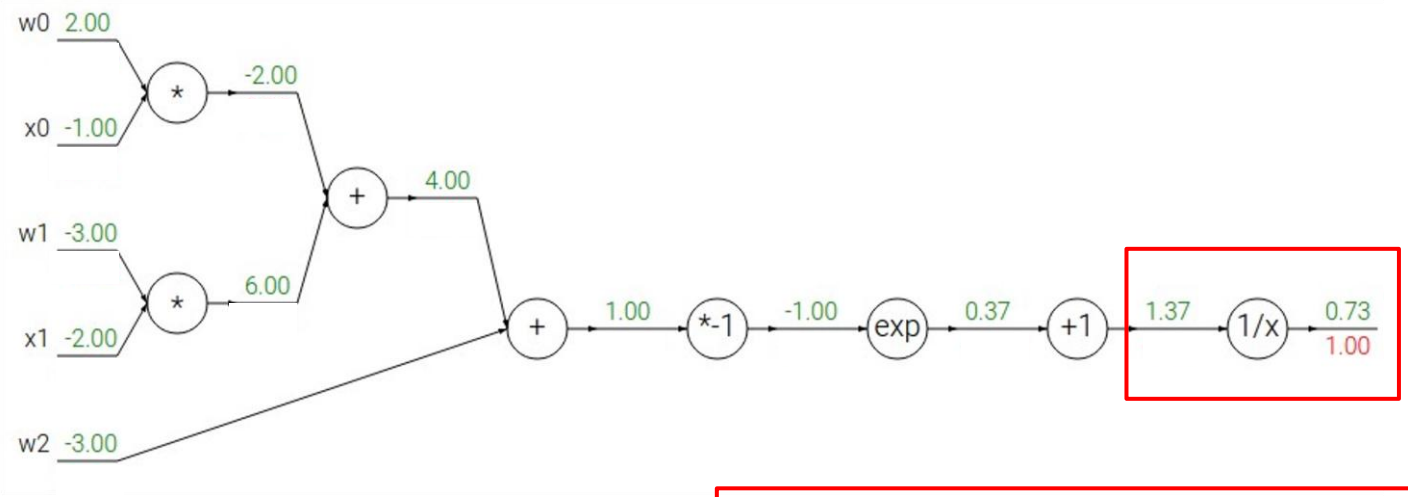
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

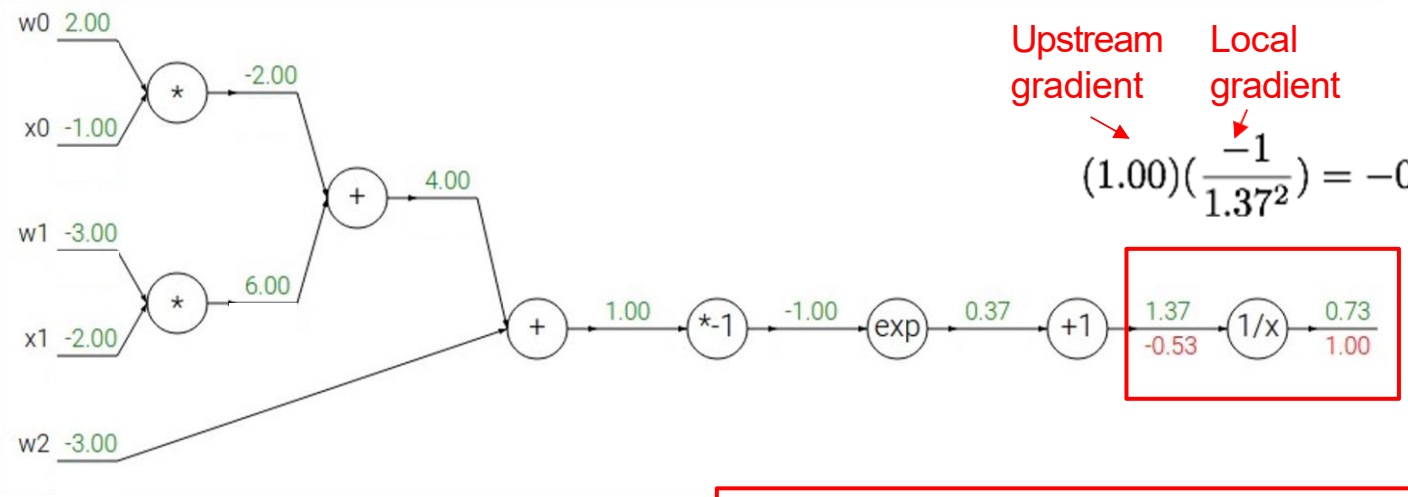
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

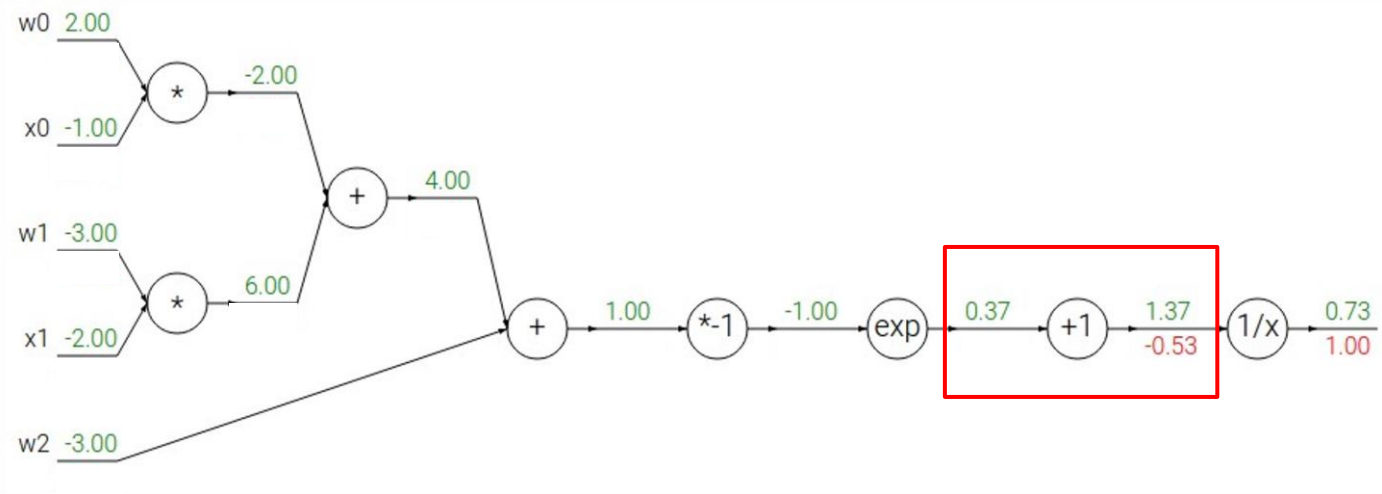
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



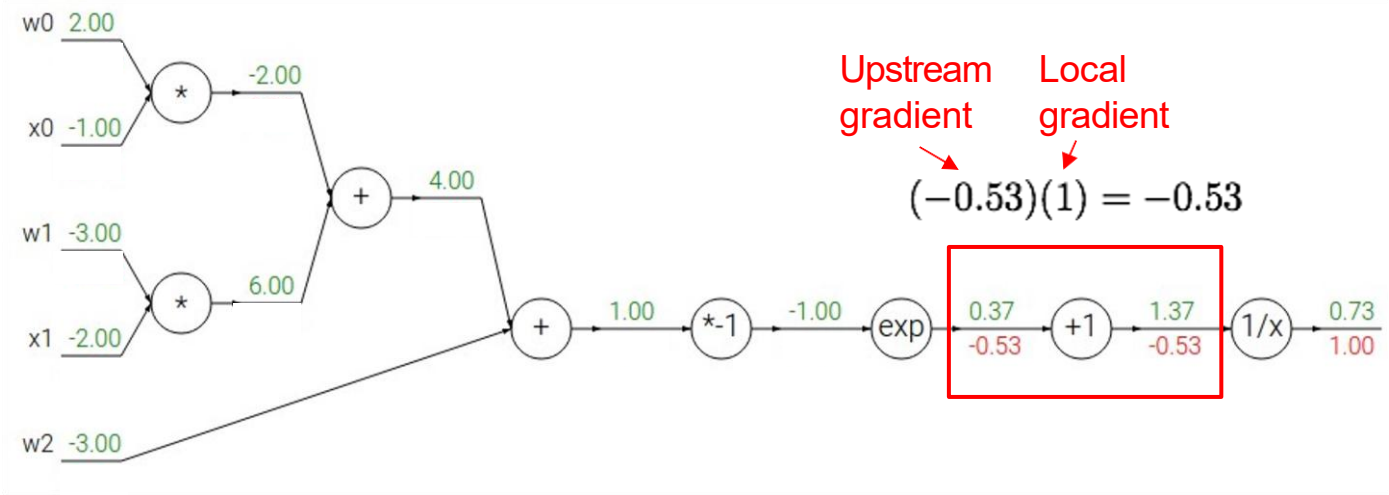
$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example: $$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

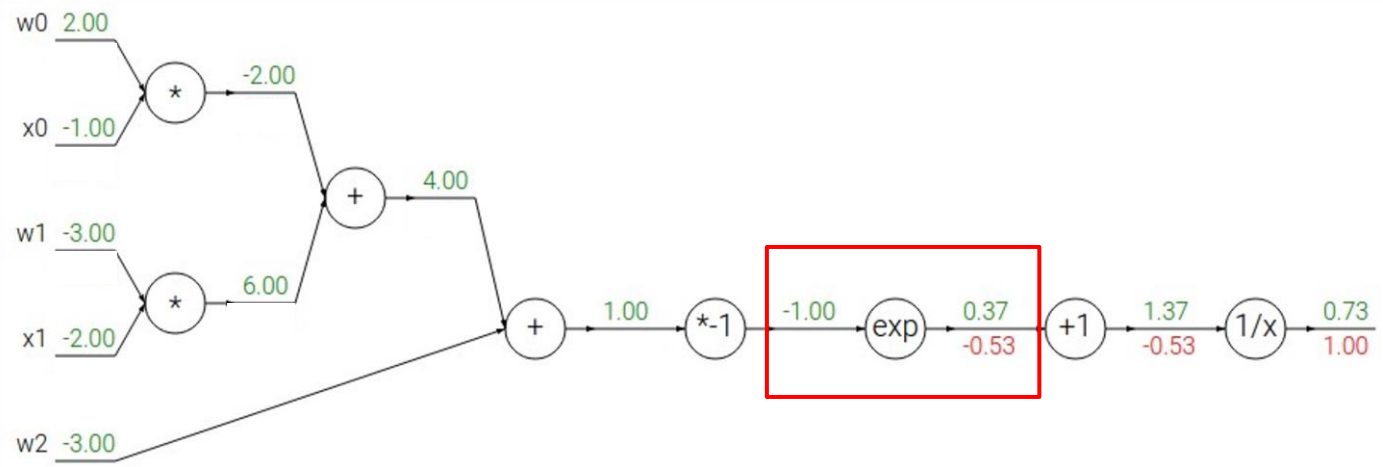
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

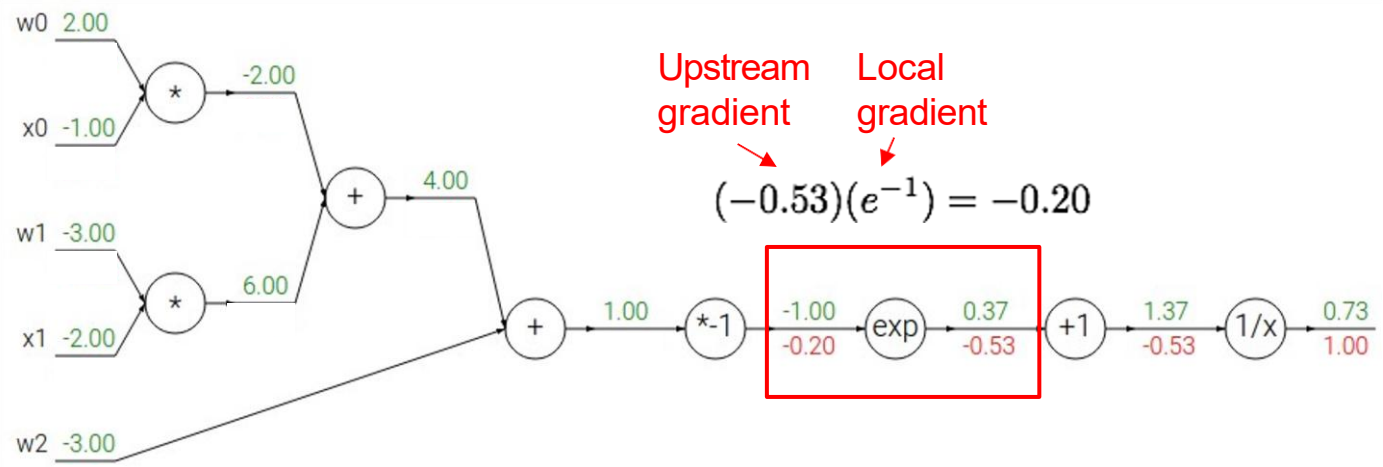
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example:

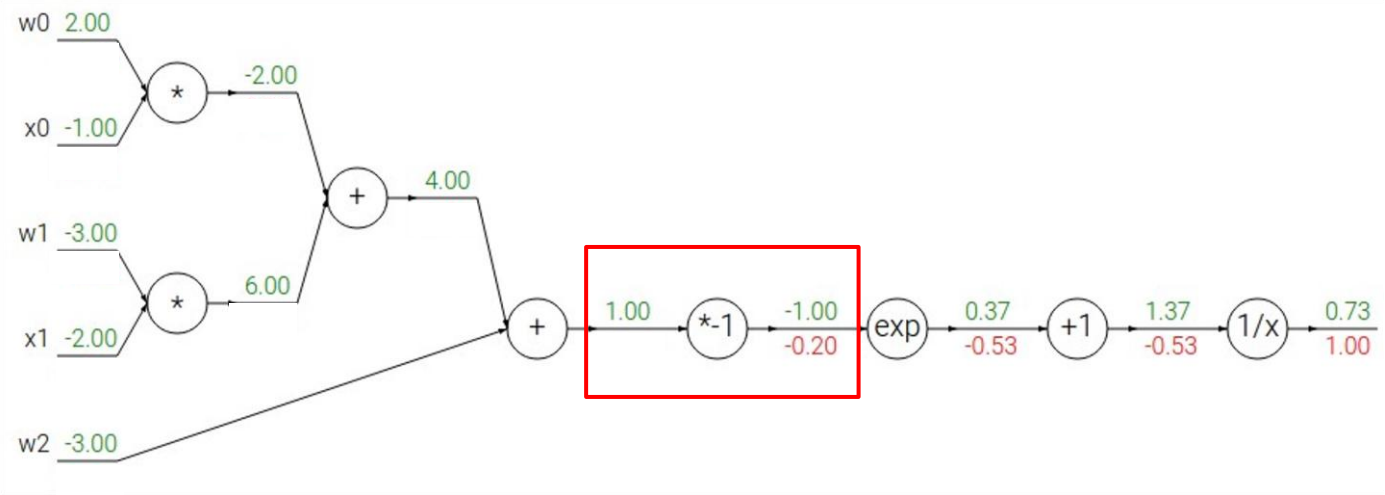
$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$	$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$	$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



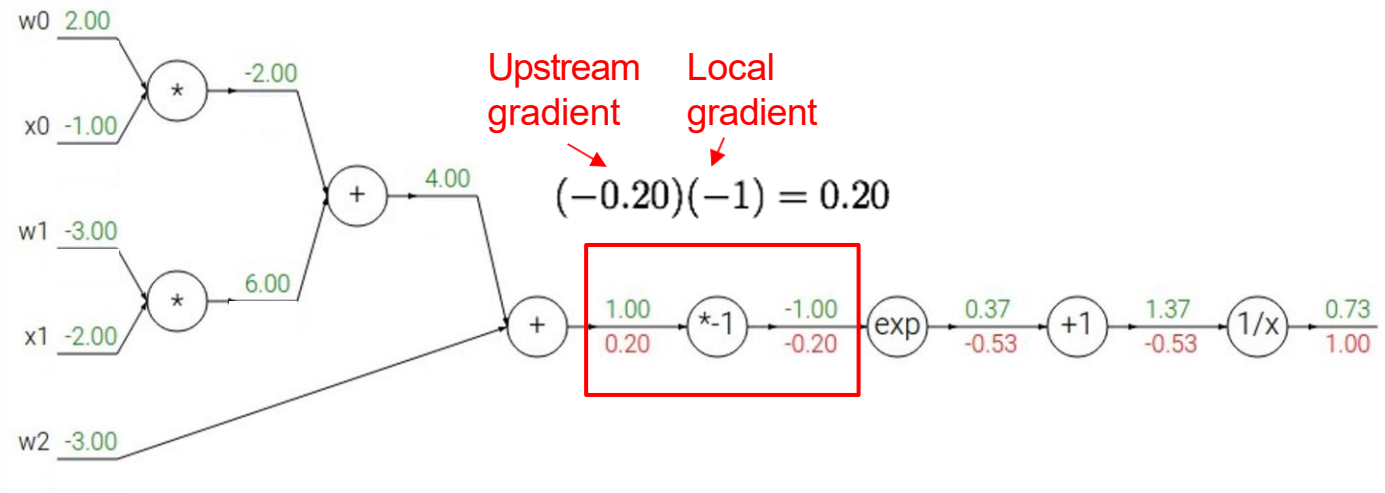
$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example: $$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

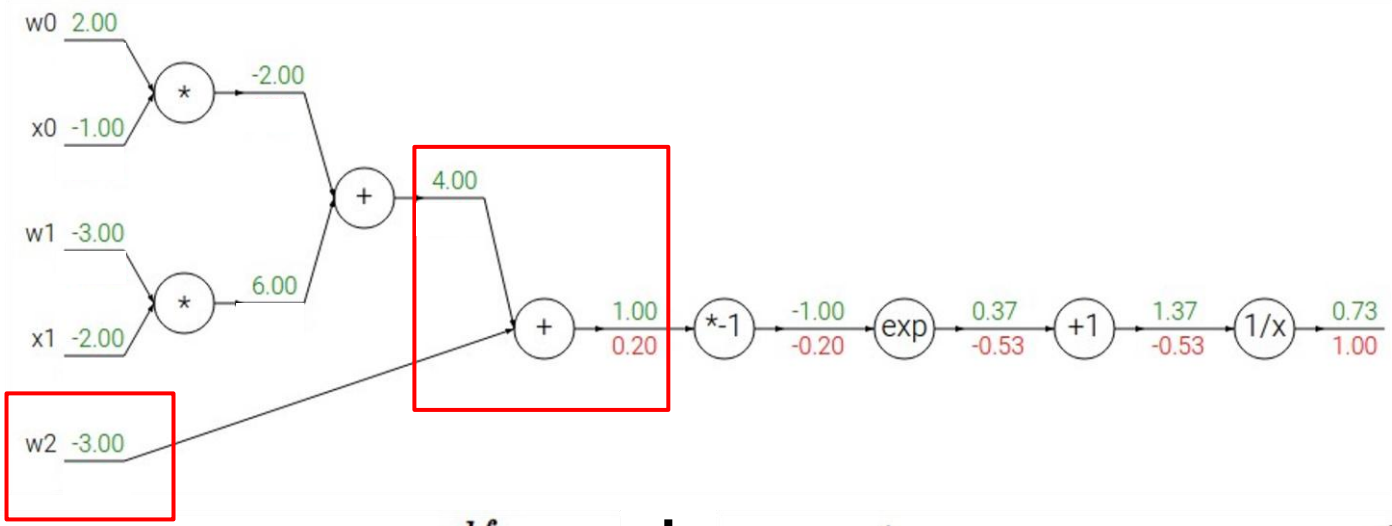
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

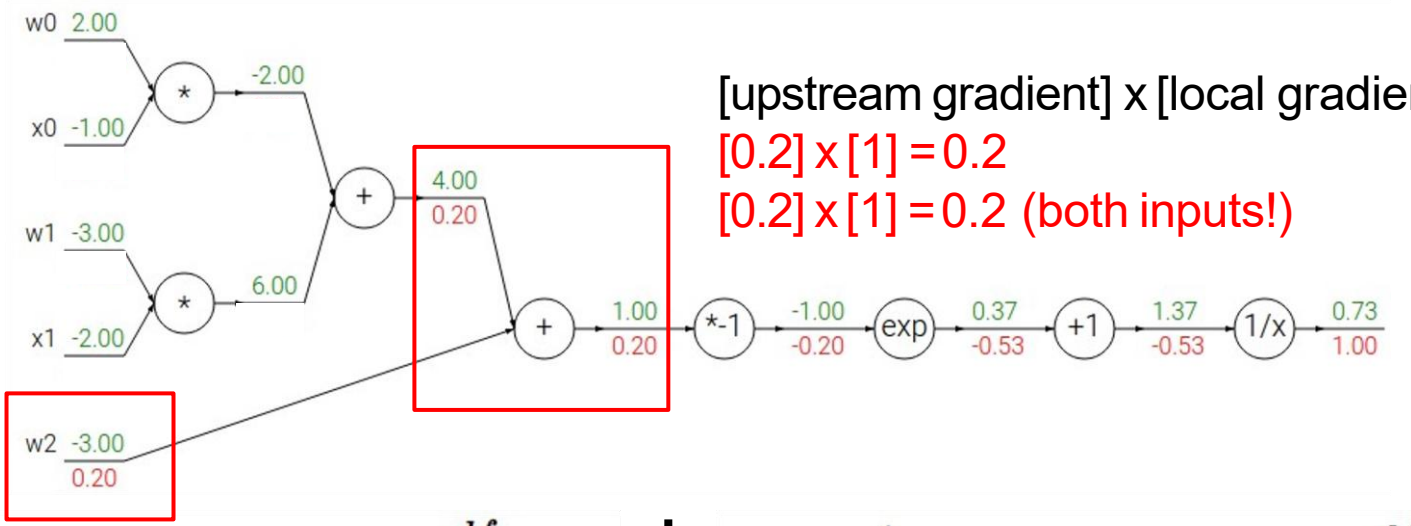
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

[0.2] x [1] = 0.2

[0.2] x [1] = 0.2 (both inputs!)

$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

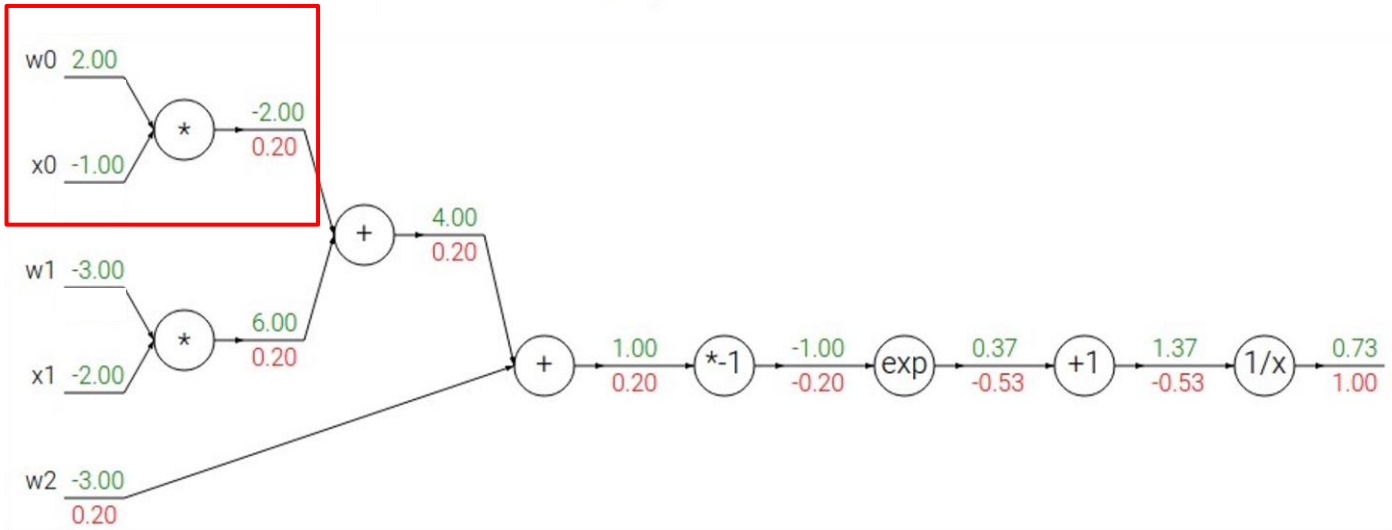
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Another example:

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

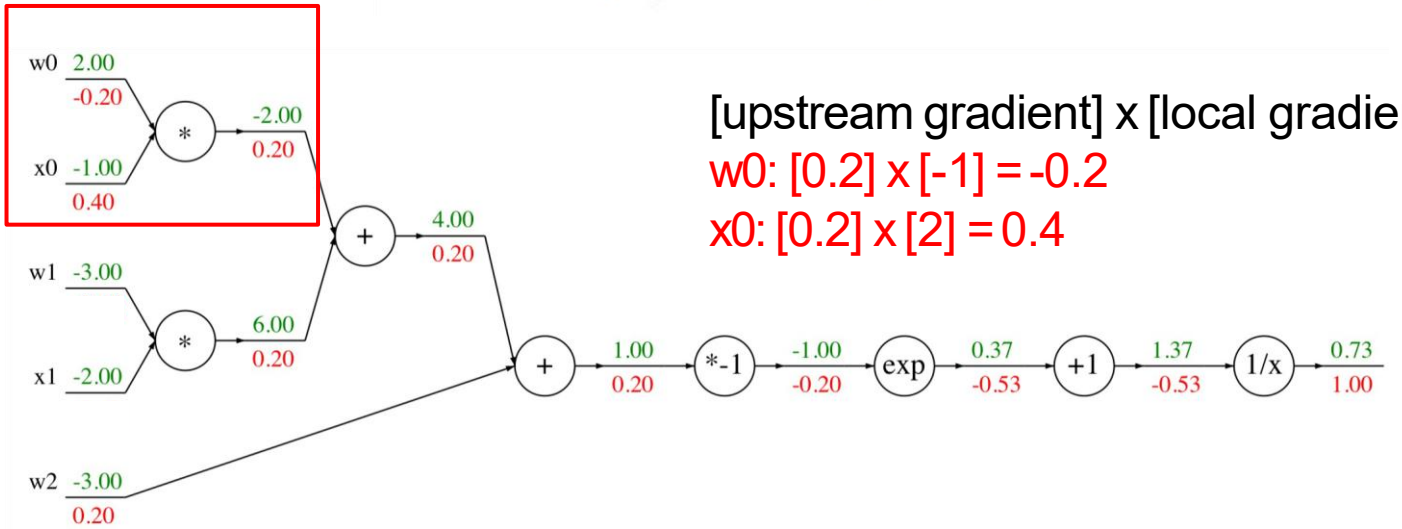
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

w0: [0.2] x [-1] = -0.2

x0: [0.2] x [2] = 0.4

$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

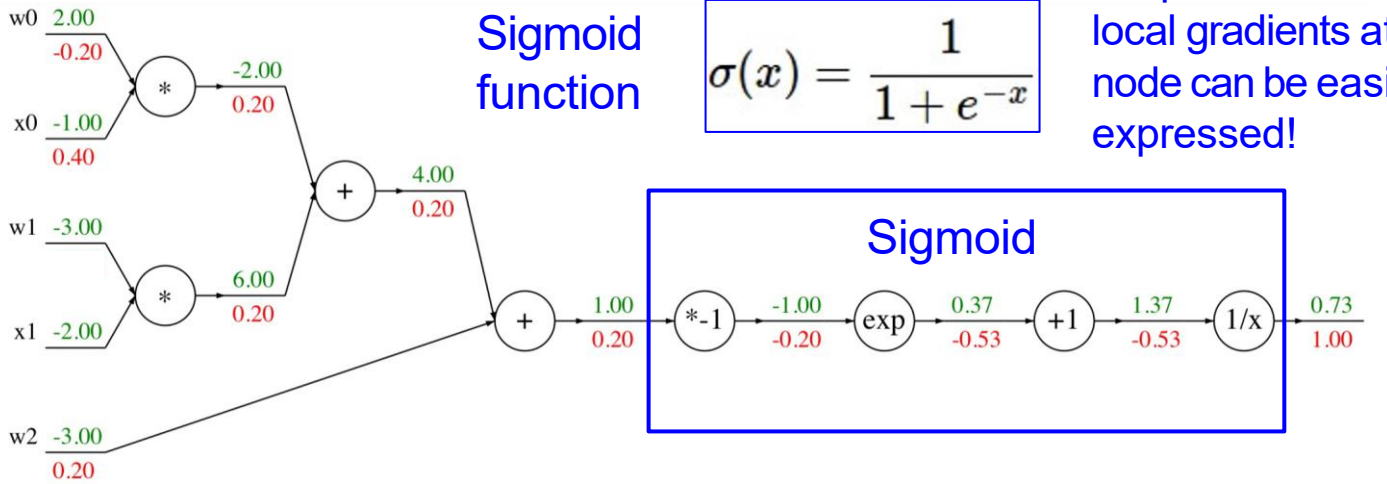
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



Sigmoid function

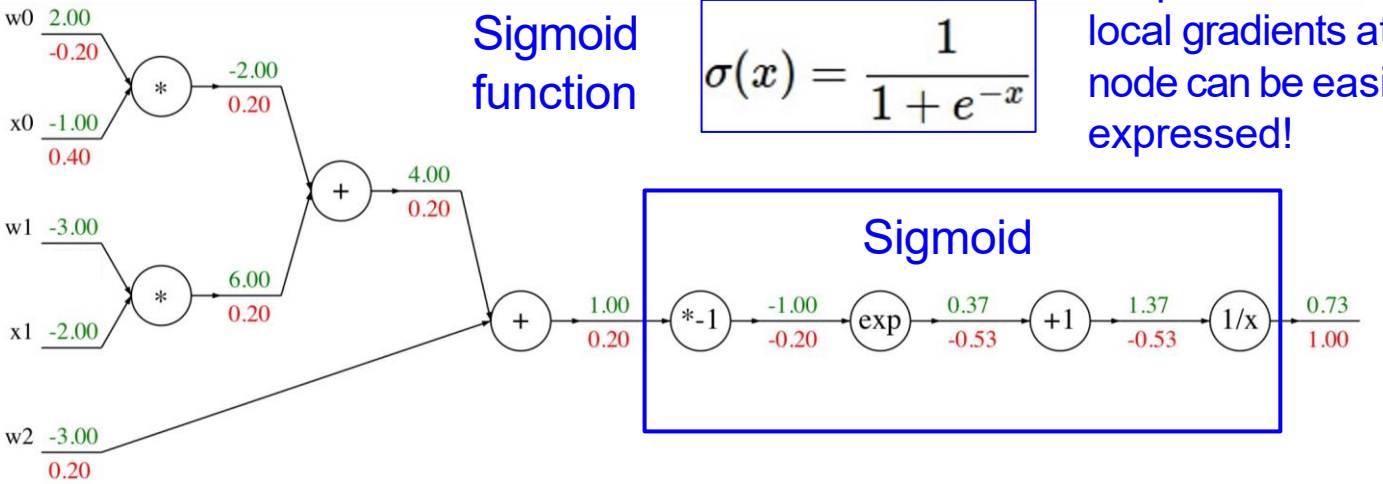
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

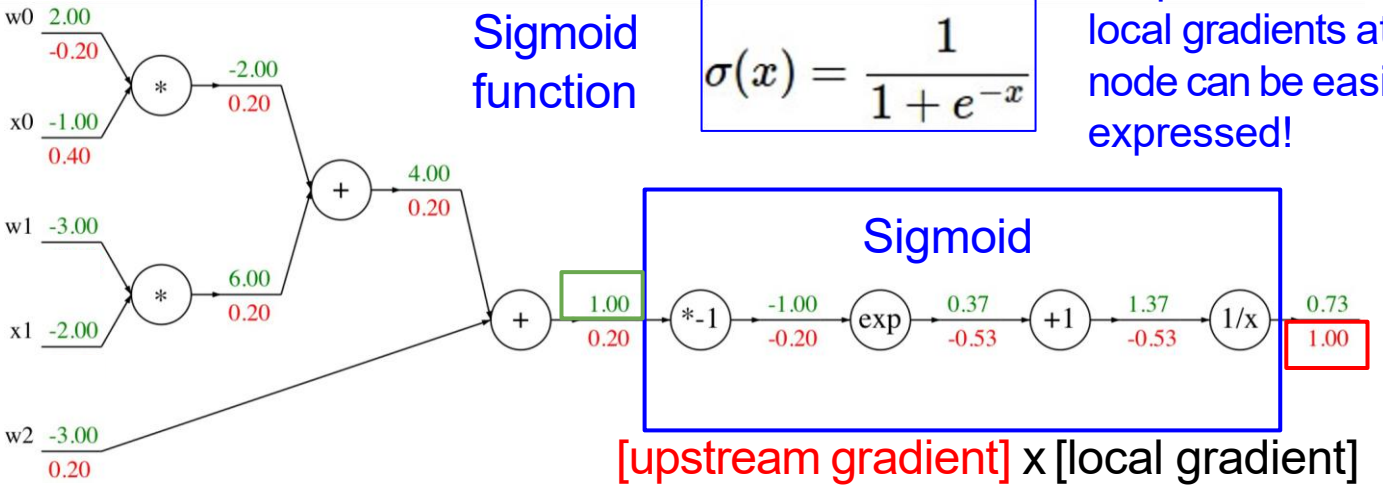
Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

[upstream gradient] x [local gradient]

[1.00] x [(1 - 1/(1+e^{-1})) (1/(1+e^{-1}))] = 0.2

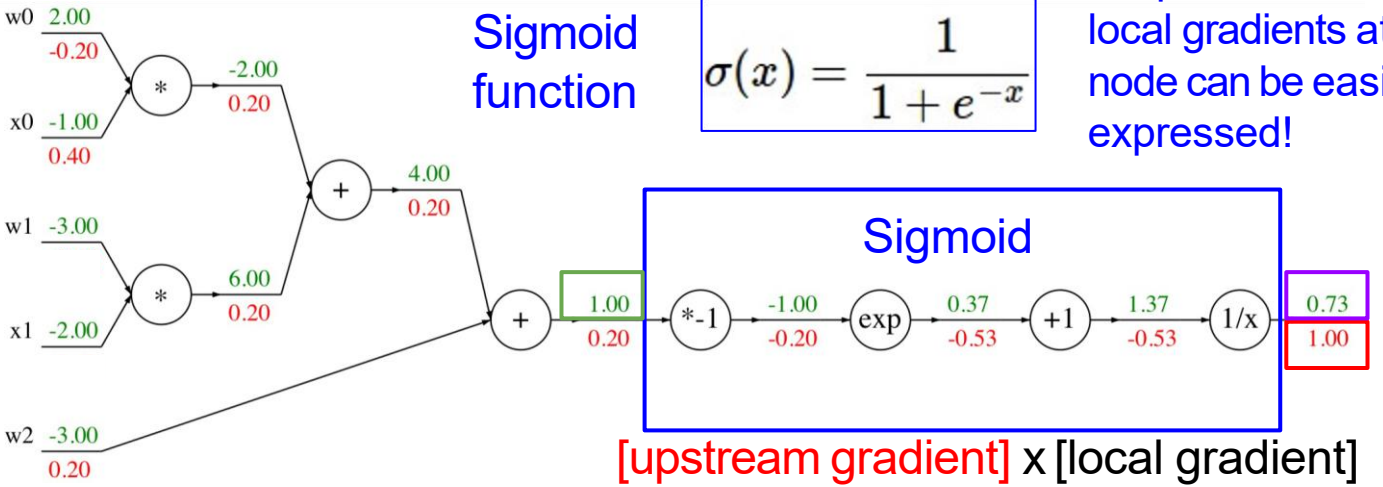
Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

[upstream gradient] x [local gradient]

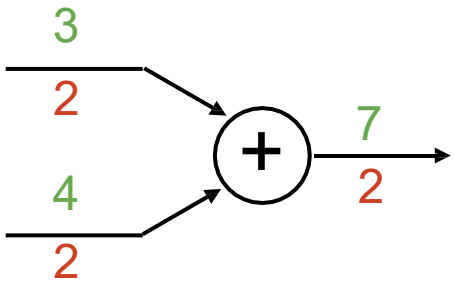
[1.00] x [(1 - 0.73) (0.73)] = 0.2

Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

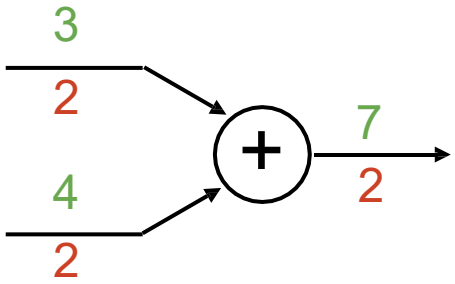
Patterns in gradient flow

add gate: gradient distributor

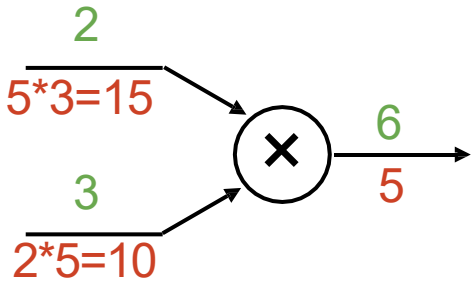


Patterns in gradient flow

add gate: gradient distributor

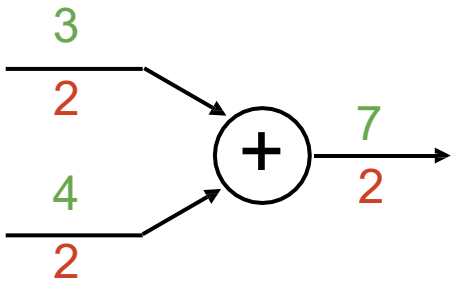


mul gate: “swap multiplier”

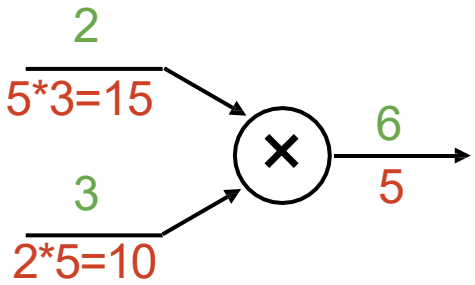


Patterns in gradient flow

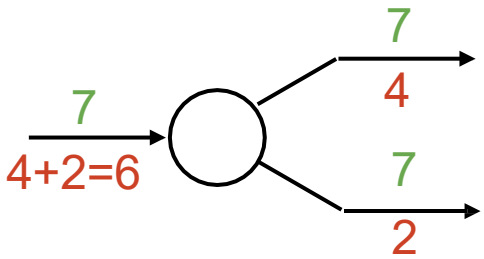
add gate: gradient distributor



mul gate: “swap multiplier”

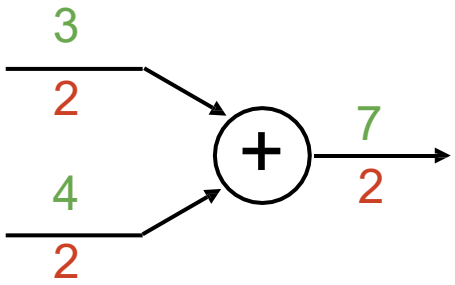


copy gate: gradient adder

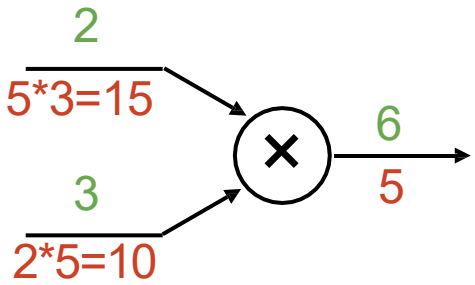


Patterns in gradient flow

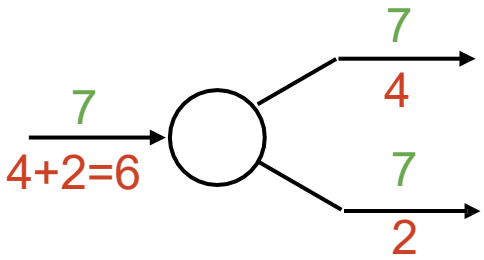
add gate: gradient distributor



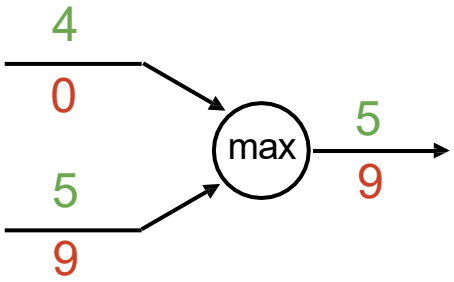
mul gate: “swap multiplier”



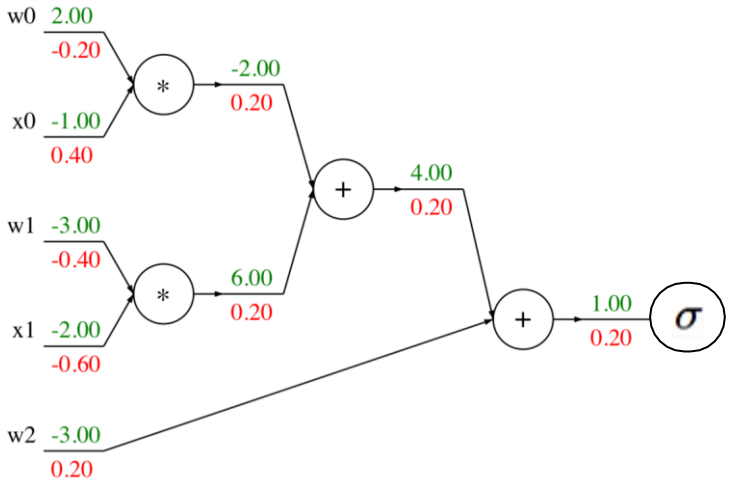
copy gate: gradient adder



max gate: gradient router



Backprop Implementation: "Flat" code



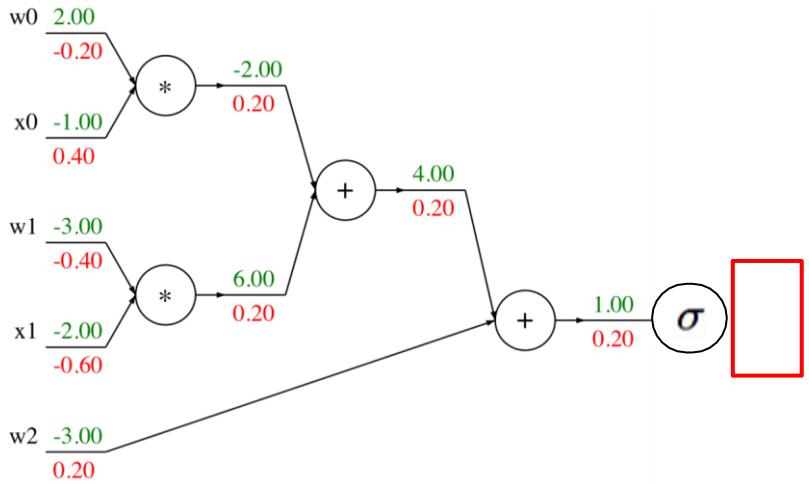
Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Backward pass:
Compute grads

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



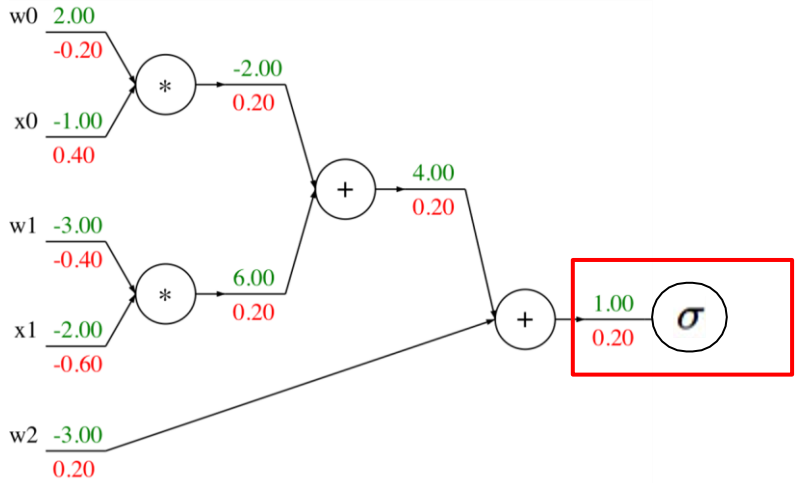
Forward pass: Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Base case

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



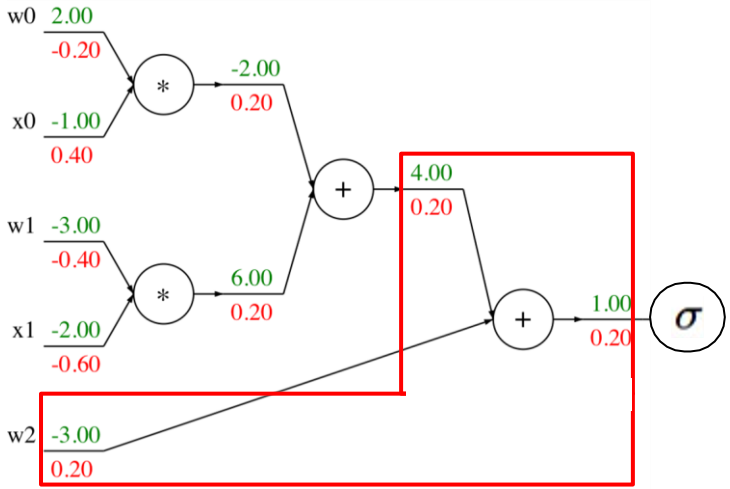
Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Sigmoid

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



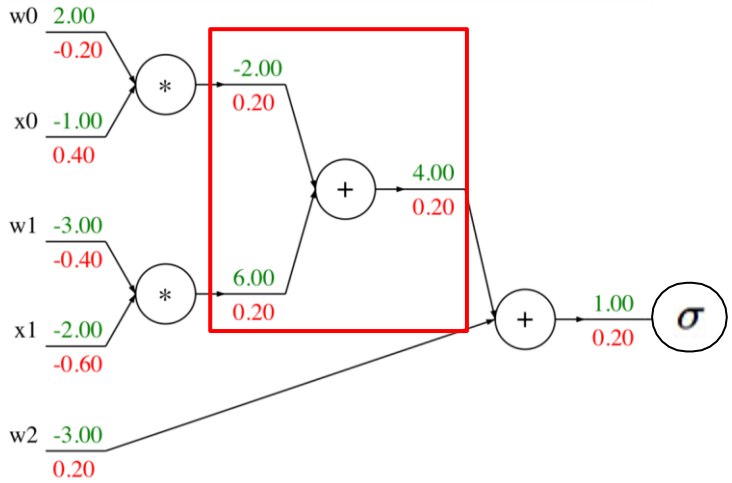
Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Add gate

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



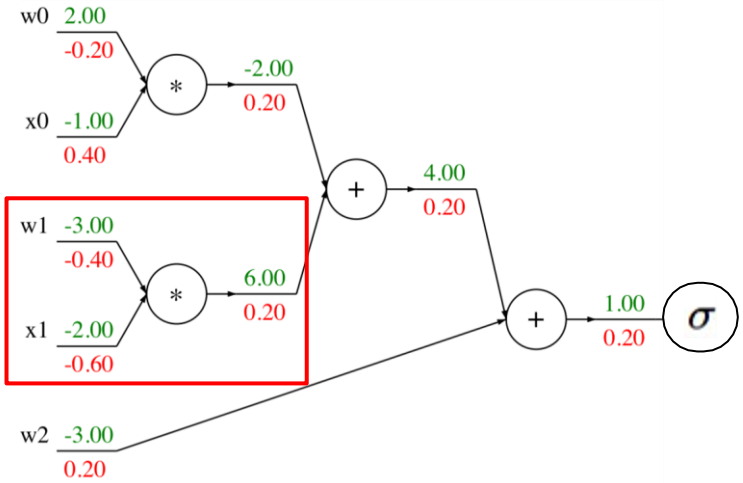
Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Add gate

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



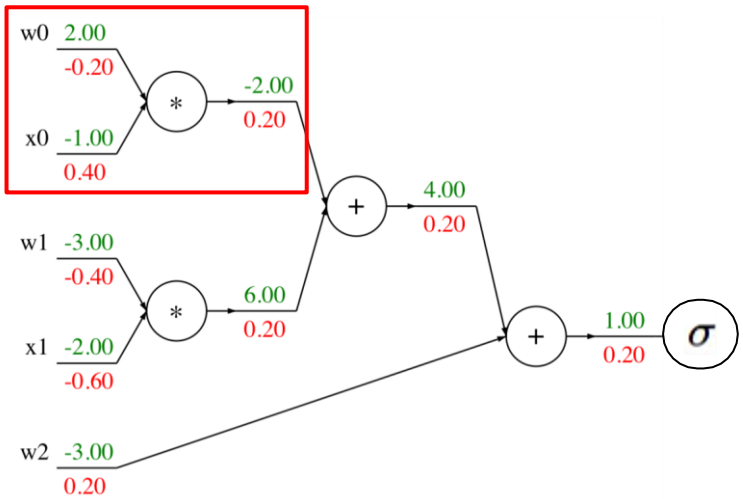
Forward pass: Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Multiply gate

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Backprop Implementation: "Flat" code



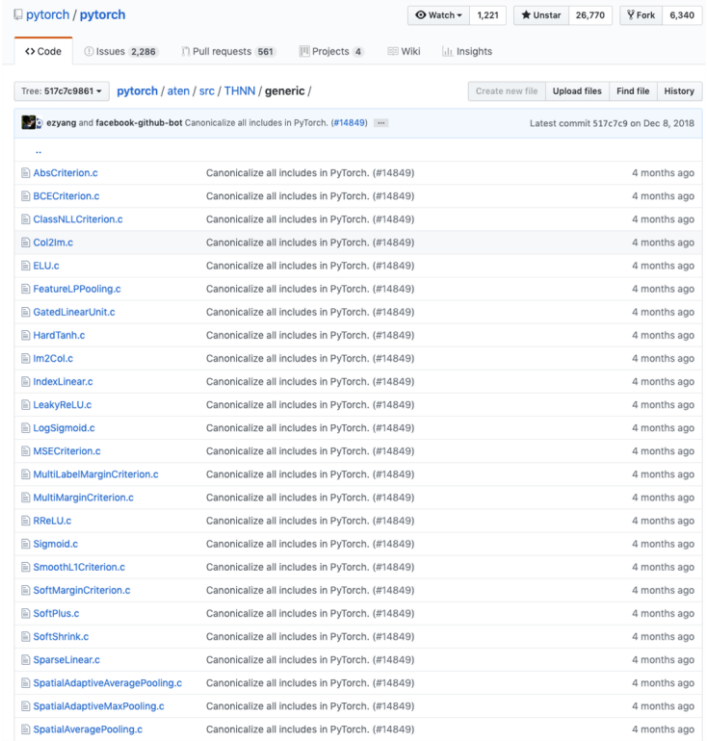
Forward pass: Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)

  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Multiply gate

Example: PyTorch operators



pytorch / pytorch

Watch 1,221 | Unstar 26,770 | Fork 6,340

Code | Issues 2,286 | Pull requests 561 | Projects 4 | Wiki | Insights

Tree: 517c7c9861 ▾ | pytorch / aten / src / THNN / generic / | Create new file | Upload files | Find file | History

ezyang and facebook-github-bot Canonicalize all includes in PyTorch. (#14849) — Latest commit 517c7c9 on Dec 8, 2018

AbsCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
BCECriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
ClassNLLCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
Col2Im.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
ELU.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
FeatureLPPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
GatedLinearUnit.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
HardTanh.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
Im2Col.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
IndexLinear.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
LeakyReLU.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
LogSigmoid.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
MSECriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
MultiLabelMarginCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
MultiMarginCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
RReLU.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
Sigmoid.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SmoothL1Criterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SoftMarginCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SoftPlus.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SoftShrink.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SparseLinear.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialAdaptiveAveragePooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialAdaptiveMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialAveragePooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialClassNLLCriterion.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialConvolutionMM.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialDilatedConvolution.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialDilatedMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialFractionalMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialFullDilatedConvolution.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialMaxUnpooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialReflectionPadding.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialReplicationPadding.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialUpSamplingBilinear.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
SpatialUpSamplingNearest.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
THNN.h	Canonicalize all includes in PyTorch. (#14849)	4 months ago
Tanh.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
TemporalReflectionPadding.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
TemporalReplicationPadding.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
TemporalRowConvolution.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
TemporalUpSamplingLinear.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
TemporalUpSamplingNearest.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricAdaptiveAveragePoolin...	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricAdaptiveMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricAveragePooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricConvolutionMM.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricDilatedConvolution.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricDilatedMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricFractionalMaxPooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricFullDilatedConvolution.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricMaxUnpooling.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricReplicationPadding.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricUpSamplingNearest.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
VolumetricUpSamplingTrilinear.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago
linear_upsampling.h	Implement nn.functional.interpolate based on upsample. (#8591)	9 months ago
pooling_shape.h	Use integer math to compute output size of pooling operations (#14405)	4 months ago
unfold.c	Canonicalize all includes in PyTorch. (#14849)	4 months ago

PyTorch sigmoid layer

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Source

PyTorch sigmoid layer

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

```
static void sigmoid_kernel(TensorIterator& iter) {
  AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
    unary_kernel_vec(
        iter,
        [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
        [=](Vec256<scalar_t> a) {
          a = Vec256<scalar_t>((scalar_t)(0)) - a;
          a = a.exp();
          a = Vec256<scalar_t>((scalar_t)(1)) + a;
          a = a.reciprocal();
          return a;
        });
  });
}
```

Forward actually defined elsewhere...

```
return (1 / (1 + std::exp((-a))));
```

Source

PyTorch sigmoid layer

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

```
static void sigmoid_kernel(TensorIterator& iter) {
  AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
    unary_kernel_vec(
        iter,
        [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
        [=](Vec256<scalar_t> a) {
          a = Vec256<scalar_t>((scalar_t)(0)) - a;
          a = a.exp();
          a = Vec256<scalar_t>((scalar_t)(1)) + a;
          a = a.reciprocal();
          return a;
        });
  });
}
```

Forward actually defined elsewhere...

Backward

$$(1-\sigma(x))\,\sigma(x)$$

Source

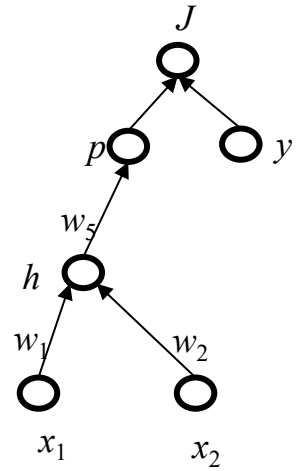
$h=f_1(h), p = f_2(p)$

$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial p}\frac{\partial p}{\partial h}$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i=\mathrm{f}_1(\mathbf{x}_i; w_1, w_2)=\max(w_1x_{1,i}+w_2x_{2,i}, 0)$
- $p_i=\mathrm{f}_2(h_i; \mathrm{w}_5)=w_5h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w})=\sum_i(p_i - y_i)^2$



$h=f_1(h), p = f_2(p)$

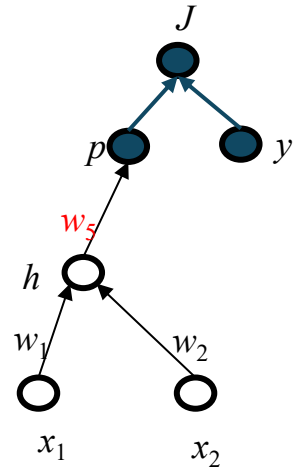
$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial p}\frac{\partial p}{\partial h}$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i=\mathrm{f}_1(\mathbf{x}_i; w_1, w_2)=\max(w_1x_{1,i}+w_2x_{2,i}, 0)$
- $p_i=\mathrm{f}_2(h_i; \mathrm{w}_5)=w_5h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w})=\sum_i(p_i - y_i)^2$

$\frac{\partial J}{\partial w_5} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial w_5}$



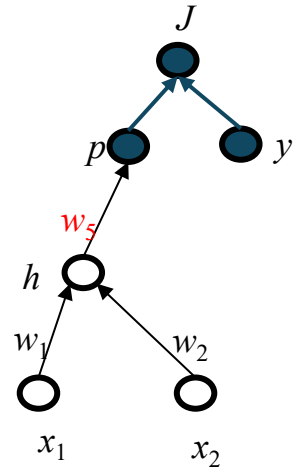
$h=f_1(h), p = f_2(p)$

$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial p}\frac{\partial p}{\partial h}$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i=\mathrm{f}_1(\mathbf{x}_i; w_1, w_2)=\max(w_1x_{1,i}+w_2x_{2,i}, 0)$
- $p_i=\mathrm{f}_2(h_i; \mathrm{w}_5)=w_5h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w})=\sum_i(p_i-y_i)^2$



$$\frac{\partial J}{\partial w_5} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial w_5}$$

$$= \sum_i 2(p_i-y_i)\frac{\partial p_i}{\partial w_5}$$

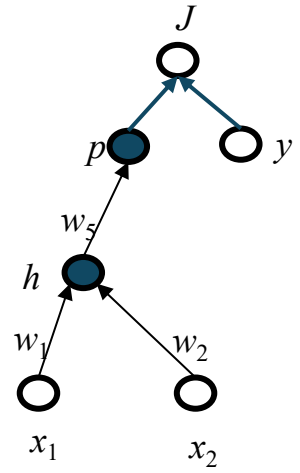
$h=f_1(h), p = f_2(p)$

$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial p}\frac{\partial p}{\partial h}$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i=\mathrm{f}_1(\mathbf{x}_i; w_1, w_2)=\max(w_1x_{1,i}+w_2x_{2,i}, 0)$
- $p_i=\mathrm{f}_2(h_i; \mathrm{w}_5)=w_5h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w})=\sum_i(p_i - y_i)^2$



$$\Delta p_i = \frac{\partial J}{\partial p_i} = 2(p_i - y_i)$$

$$\frac{\partial J}{\partial w_5} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial w_5}$$
$$= \sum_i 2(p_i - y_i)\frac{\partial p_i}{\partial w_5}$$
$$= \sum_i 2(p_i - y_i)h_i$$

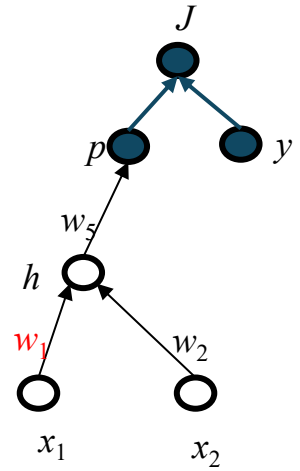
$h=f_1(h), p = f_2(p)$

$\frac{\partial J}{\partial h} = \frac{\partial J}{\partial p}\frac{\partial p}{\partial h}$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i=\mathrm{f}_1(\mathbf{x}_i; w_1, w_2)=\max(w_1 x_{1,i}+w_2 x_{2,i}, 0)$
- $p_i=\mathrm{f}_2(h_i; \mathrm{w}_5)=w_5 h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w})=\sum_i(p_i - y_i)^2$



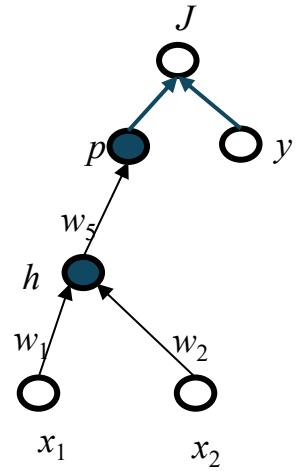
$$\frac{\partial J}{\partial w_1} = \sum_i \boxed{\frac{\partial J}{\partial p_i}} \frac{\partial p_i}{\partial w_1} = \sum_i \boxed{2(p_i - y_i)} \frac{\partial p_i}{\partial w_1} = \sum_i \boxed{\Delta p_i} \frac{\partial p_i}{\partial w_1}$$

$$\Delta p_i = \frac{\partial J}{\partial p_i} = 2(p_i - y_i)$$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i = \mathrm{f}_1(\mathbf{x}_i; w_1, w_2) = \max(w_1 x_{1,i} + w_2 x_{2,i}, 0)$
- $p_i = \mathrm{f}_2(h_i; \mathrm{w}_5) = w_5 h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1}$$
$$= \sum_i \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial w_1}$$
$$= \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1}$$

$$\Delta p_i = \frac{\partial J}{\partial p_i} = 2(p_i - y_i)$$

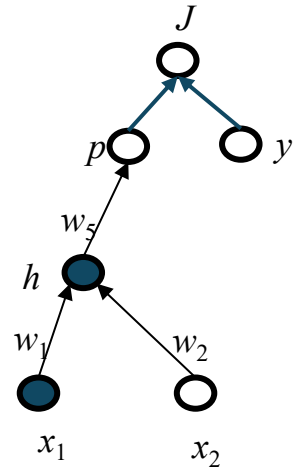
$$\Delta h_i = \frac{\partial J}{\partial h_i} = \Delta p_i \frac{\partial p_i}{\partial h_i} = 2(p_i - y_i) w_5$$

$u(x) = 1, if\ x > 0,$
$u(x) = 0, if\ x \leq 0.$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i = \mathrm{f}_1(\mathbf{x}_i; w_1, w_2) = \max(w_1 x_{1,i} + w_2 x_{2,i}, 0)$
- $p_i = \mathrm{f}_2(h_i; \mathrm{w}_5) = w_5 h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial h_i}\frac{\partial h_i}{\partial w_1} = \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1} = \sum_i \Delta h_i\ u(w_1 x_{1,i} + w_2 x_{2,i}) x_{1,i}$$

$$\Delta p_i = \frac{\partial J}{\partial p_i} = 2(p_i - y_i)$$

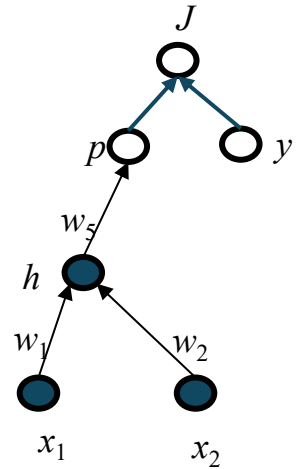
$$\Delta h_i = \frac{\partial J}{\partial h_i} = \Delta p_i \frac{\partial p_i}{\partial h_i} = 2(p_i - y_i) w_5$$

$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \Delta h_i \frac{\partial h_i}{\partial x_{1,i}} = 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) w_1$$

$u(x) = 1, if\ x > 0,$
$u(x) = 0, if\ x \leq 0.$

Back-propagation

- A chaining rule for obtaining gradient
- $h_i = \mathrm{f}_1(\mathbf{x}_i; w_1, w_2) = \max(w_1 x_{1,i} + w_2 x_{2,i}, 0)$
- $p_i = \mathrm{f}_2(h_i; \mathrm{w}_5) = w_5 h_i$



- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$

$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial h_i}\frac{\partial h_i}{\partial w_1} = \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1} = \sum_i 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) x_{1,i}$$

$$\Delta h_i = 2(p_i - y_i) w_5$$

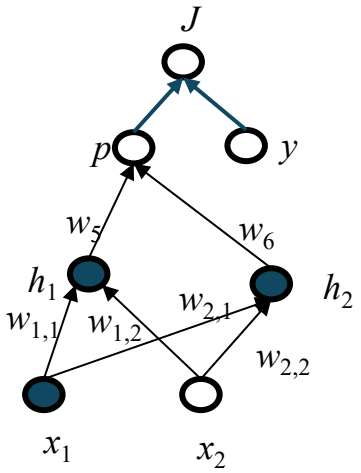
$$\frac{\partial J}{\partial w_2} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_2} = \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial h_i}\frac{\partial h_i}{\partial w_2} = \sum_i \Delta h_i \frac{\partial h_i}{\partial w_2} = \sum_i 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) x_{2,i}$$

$u(x) = 1, if\ x > 0,$
$u(x) = 0, if\ x \leq 0.$

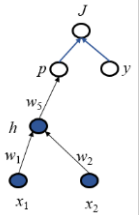
Back-propagation

- A chaining rule for obtaining gradient
- $h_i = \mathrm{f}_1(\mathbf{x}_i; w_1, w_2) = \max(w_1 x_{1,i} + w_2 x_{2,i}, 0)$
- $p_i = \mathrm{f}_2(h_i; \mathrm{w}_5) = w_5 h_i$

- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



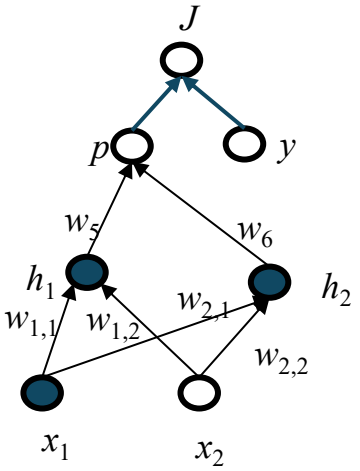
$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1}$$
$$= \sum_i \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial w_1}$$
$$= \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1}$$
$$= \sum_i 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) x_{1,i}$$



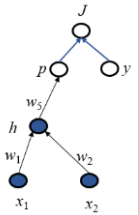
$u(x) = 1, if\ x > 0,$
$u(x) = 0, if\ x \leq 0.$

Back-propagation

- A chaining rule for obtaining gradient
- $h_{1,i} = \max(w_{1,1}x_{1,i} + w_{1,2}x_{2,i}, 0)$
- $h_{2,i} = \max(w_{2,1}x_{1,i} + w_{2,2}x_{2,i}, 0)$
- $p_i = f_2(\mathbf{h}_i; w_3) = w_5 h_{1,i} + w_6 h_{2,i}$
- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1}$$
$$= \sum_i \frac{\partial J}{\partial p_i}\frac{\partial p_i}{\partial h_i}\frac{\partial h_i}{\partial w_1}$$
$$= \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1}$$
$$= \sum_i 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) x_{1,i}$$



$\frac{\partial J}{\partial w_{1,1}} = ?$

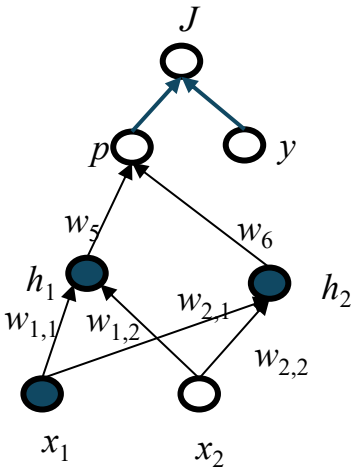
$\frac{\partial J}{\partial x_{1,i}} = ?$

$$\Delta h_{1,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_5 \qquad u(x) = 1, if\ x > 0, \quad u(x) = 0, if\ x \leq 0.$$

Back-propagation

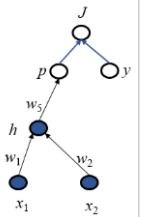
$$\Delta h_{2,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_6$$

- A chaining rule for obtaining gradient
- $h_{1,i} = \max(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}, 0)$
- $h_{2,i} = \max(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}, 0)$
- $p_i = f_2(\mathbf{h}_i; w_3) = w_5 h_{1,i} + w_6 h_{2,i}$
- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_{j,k}} = \sum_i \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial w_{j,k}}$$

$$\frac{\partial J}{\partial w_1} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_1} = \sum_i \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial w_1} = \sum_i \Delta h_i \frac{\partial h_i}{\partial w_1} = \sum_i 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) x_{1,i}$$



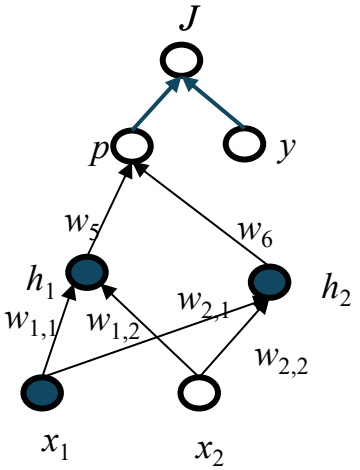
$$\frac{\partial J}{\partial w_{1,1}} = \sum_i \Delta p_i \frac{\partial p_i}{\partial w_{1,1}} = \sum_i \sum_j \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_{j,i}} \frac{\partial h_{j,i}}{\partial w_{1,1}} = \sum_i \sum_j \Delta h_{j,i} \frac{\partial h_{j,i}}{\partial w_{1,1}} = \sum_i 2(p_i - y_i)[w_5 u(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}) x_{1,i}]$$

$$\Delta h_{1,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_5 \qquad u(x) = 1, if\ x > 0, \quad u(x) = 0, if\ x \le 0.$$

Back-propagation

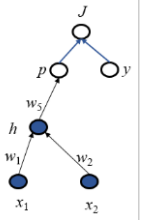
$$\Delta h_{2,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_6$$

- A chaining rule for obtaining gradient
- $h_{1,i} = \max(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}, 0)$
- $h_{2,i} = \max(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}, 0)$
- $p_i = f_2(\mathbf{h}_i; w_3) = w_5 h_{1,i} + w_6 h_{2,i}$
- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_{j,k}} = \sum_i \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial w_{j,k}}$$

$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial x_{1,i}}$$
$$= \Delta h_i \frac{\partial h_i}{\partial x_{1,i}}$$
$$= 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) w_1$$



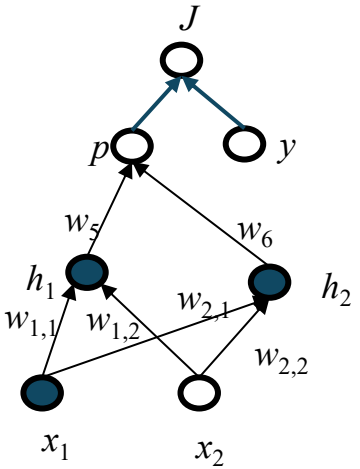
$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \sum_i \sum_j \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_{j,i}} \frac{\partial h_{j,i}}{\partial x_{1,i}}$$
$$= \sum_j \Delta h_{j,i} \frac{\partial h_{j,i}}{\partial x_{1,i}}$$
$$= 2(p_i - y_i)\, w_5\, u(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}) w_{1,1}$$
$$+ 2(p_i - y_i)\, w_6\, u(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}) w_{2,1}$$

$$\Delta h_{1,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_5 \qquad u(x) = 1, if\ x > 0, \quad u(x) = 0, if\ x \le 0.$$

Back-propagation

$$\Delta h_{2,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_6$$

- A chaining rule for obtaining gradient
- $h_{1,i} = \max(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}, 0)$
- $h_{2,i} = \max(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}, 0)$
- $p_i = f_2(\mathbf{h}_i; w_3) = w_5 h_{1,i} + w_6 h_{2,i}$
- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_{j,k}} = \sum_i \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial w_{j,k}}$$

$$\Delta x_{k,i} = \sum_j \Delta h_{j,i} \frac{\partial h_{j,i}}{\partial x_{k,i}}$$

$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \sum_j \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_{j,i}} \frac{\partial h_{j,i}}{\partial x_{1,i}}$$

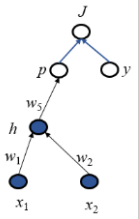
$$= \sum_j \Delta h_{j,i} \frac{\partial h_{j,i}}{\partial x_{1,i}}$$

$$= 2(p_i - y_i) w_5 u(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}) w_{1,1} + 2(p_i - y_i) w_6 u(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}) w_{2,1}$$

$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial x_{1,i}}$$

$$= \Delta h_i \frac{\partial h_i}{\partial x_{1,i}}$$

$$= 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) w_1$$

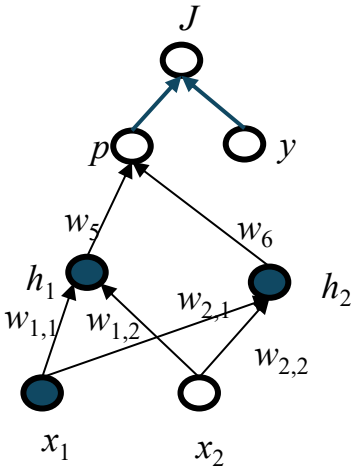


$$\Delta h_{1,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_5 \qquad u(x) = 1, if\ x > 0, \quad u(x) = 0, if\ x \le 0.$$

Back-propagation

$$\Delta h_{2,i} = \Delta p_i \frac{\partial p_i}{\partial h_{j,i}} = 2(p_i - y_i) w_6$$

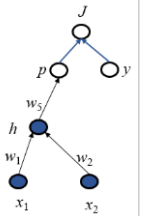
- A chaining rule for obtaining gradient
- $h_{1,i} = \max(w_{1,1} x_{1,i} + w_{1,2} x_{2,i}, 0)$
- $h_{2,i} = \max(w_{2,1} x_{1,i} + w_{2,2} x_{2,i}, 0)$
- $p_i = f_2(\mathbf{h}_i; w_3) = w_5 h_{1,i} + w_6 h_{2,i}$
- Loss:
- $J(\mathbf{p}, \mathbf{w}) = \sum_i (p_i - y_i)^2$



$$\frac{\partial J}{\partial w_{j,k}} = \sum_i \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial w_{j,k}}$$

$$\Delta x_{k,i} = \sum_j \Delta h_{j,i} \frac{\partial h_{j,i}}{\partial x_{k,i}}$$

$$\Delta x_{1,i} = \frac{\partial J}{\partial x_{1,i}} = \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial h_i} \frac{\partial h_i}{\partial x_{1,i}}$$
$$= \Delta h_i \frac{\partial h_i}{\partial x_{1,i}}$$
$$= 2(p_i - y_i) w_5 u(w_1 x_{1,i} + w_2 x_{2,i}) w_1$$



General form:

$$\frac{\partial J}{\partial w_{j,k}} = \sum_i \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial w_{j,k}}$$

$$\Delta h^{l-1}_{k,i} = \sum_j \Delta h^l_{j,i} \frac{\partial h^l_{j,i}}{\partial h^{l-1}_{k,i}}$$

Summary for today:

- (Fully-connected) Neural Networks are stacks of linear functions and nonlinear activation functions; they have much more representational power than linear classifiers
- backpropagation = recursive application of the chain rule along a computational graph to compute the gradients of all inputs/parameters/intermediates
- implementations maintain a graph structure, where the nodes implement the forward() / backward() API
- forward: compute result of an operation and save any intermediates needed for gradient computation in memory
- backward: apply the chain rule to compute the gradient of the loss function with respect to the inputs